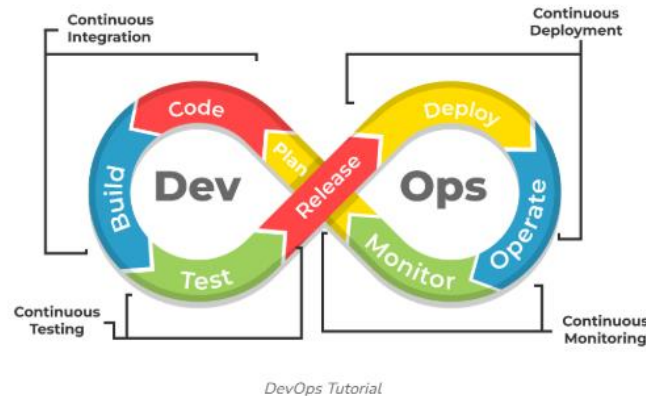


DEVOPS

DevOps is a collection of two words, “**Development**” and “**Operations**,” representing a cultural approach that emphasizes collaboration between development and operations teams to streamline the entire software delivery lifecycle.

This **DevOps tutorial** is designed to help learn DevOps basics and advanced concepts, including Git, Ansible, Docker, Puppet, Jenkins, a range of DevOps tools, Azure DevOps, Chef, Nagios, and Kubernetes.



What is DevOps?

DevOps is a software development approach emphasizing collaboration, automation, and continuous delivery to provide high-quality products to customers quickly and efficiently. DevOps breaks down silos between development and operations teams to enable seamless communication, faster time-to-market, and improved customer satisfaction.

It allows a team to handle the complete application lifecycle, from development to testing, operations, and deployment. It shows cooperation between Development and Operations groups to deploy code to production quickly in an automated and repeatable manner.

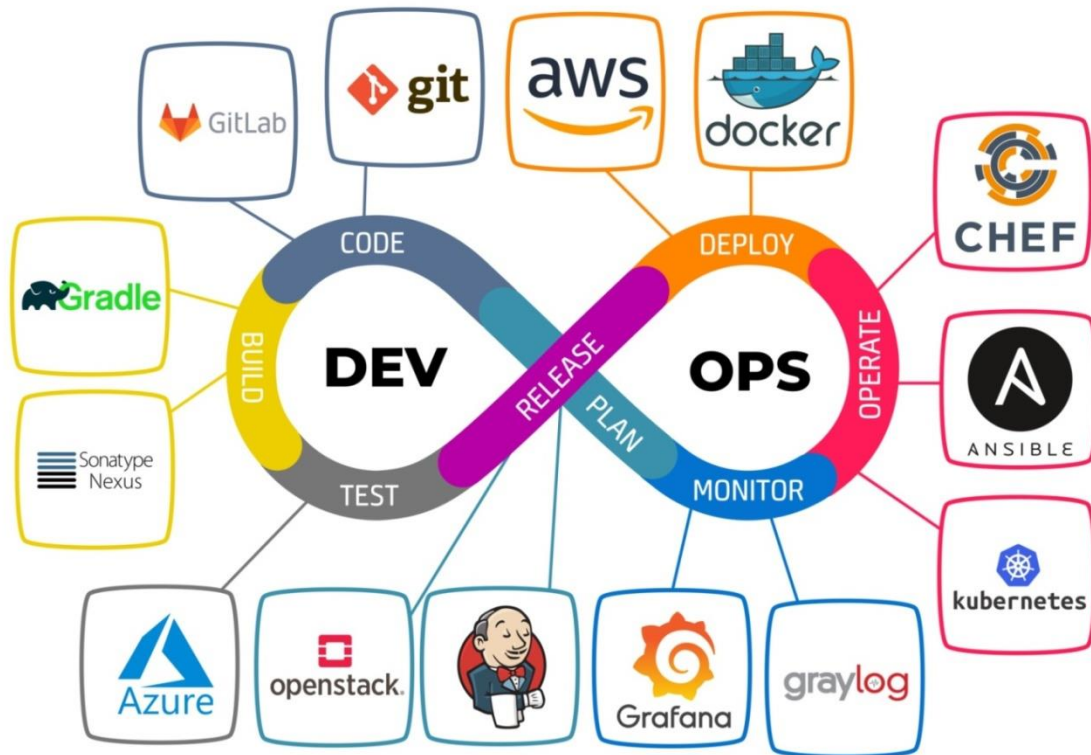
Every phase of the software development lifecycle, including planning, coding, testing, deployment, and monitoring, is heavily automated in DevOps. This improves productivity, ensures consistency, and lowers error rates in the development process. A culture of continuous improvement is also promoted by DevOps, where feedback loops are incorporated into the procedure to facilitate quicker iteration and better decision-making. Organizations can increase their agility, lower costs, and speed up innovation by adopting DevOps.

Need for DevOps:

DevOps is essential for modern software development as it integrates development and IT operations to enhance collaboration, efficiency, and quality. By implementing practices like continuous integration and continuous delivery (CI/CD), organizations can accelerate their time to market, improve software quality through automated testing, and foster a culture of shared

responsibility among teams. This leads to faster feedback loops, increased scalability, and better security.

DevOps Tools:



Experiment-1

1. Introduction to Maven and Gradle: Overview of build automation tools, Key differences between Maven and Gradle, Installation and Setup.

Overview of Build Automation Tools

What is Build Automation?

Build automation is the process of scripting or automating the compilation, testing, packaging, and deployment of software projects. It is a crucial part of **Continuous Integration (CI)** and **Continuous Deployment (CD)** in DevOps. It makes development more efficient and error-free.

Why Use Build Automation Tools?

- Reduces manual errors in the build process.
- Increases productivity by automating repetitive tasks.
- Ensures consistency in software builds.
- Facilitates continuous integration and deployment.

Popular Build Automation Tools

- **Maven** (Apache Maven)
- **Gradle**
- **Ant** (Apache Ant)
- **Bazel** (by Google)
- **Make** (GNU Make)

Among these, **Maven and Gradle** are the most widely used in **Java-based** projects. Both are great for managing project builds and dependencies, but they have some key differences.

Introduction to Maven

What is Maven?

Apache Maven is a Java-based build automation and project management tool that follows a convention-over-configuration approach.

It uses an XML configuration file called pom.xml (Project Object Model) to define project settings, dependencies, and build steps.

Features of Maven

- Uses **XML-based configuration** (pom.xml).
- Supports dependency management via the **Maven Central Repository**.
- Automates the build lifecycle (compile, test, package, deploy).
- Integrates with tools like **Jenkins, SonarQube, Docker, and Kubernetes**.

Maven Build Lifecycle

Maven follows a structured build lifecycle with phases such as:

1. **validate** – Validates project configuration.
2. **compile** – Compiles the source code.
3. **test** – Runs unit tests.
4. **package** – Packages code (e.g., jar or war).
5. **install** – Installs package locally.
6. **deploy** – Deploys artifact to a remote repository.

Introduction to Gradle

What is Gradle?

Gradle is a powerful, flexible, and high-performance build tool that supports **multi-language builds** (Java, Kotlin, Groovy, C++, etc.). It uses a domain-specific language (DSL) for build scripts, written in Groovy or Kotlin.

Features of Gradle

- Uses **Groovy or Kotlin DSL** instead of XML (build.gradle).
- Faster than Maven due to **incremental builds**.
- Supports **dependency management** using repositories like Maven Central.
- Integrates with tools like **Jenkins, Docker, Kubernetes, and Android Studio**.

Gradle Build Lifecycle

Gradle doesn't follow a fixed lifecycle like Maven but is highly customizable using tasks:

1. **clean** – Deletes previous build files.
2. **build** – Compiles and packages the project.
3. **test** – Runs unit tests.
4. **assemble** – Creates executable artifacts.

Key Differences Between Maven and Gradle

Feature	Maven	Gradle
Configuration	Uses XML (pom.xml)	Uses Groovy/Kotlin (build.gradle)
Performance	Slower due to full builds	Faster due to incremental builds
Dependency Management	Centralized with Maven Central	Flexible dependency management
Learning Curve	Easier due to convention-over-configuration	Steeper due to flexibility
Used By	Java, Spring Boot projects	Java, Android, Kotlin projects
Customization	Limited	Highly customizable

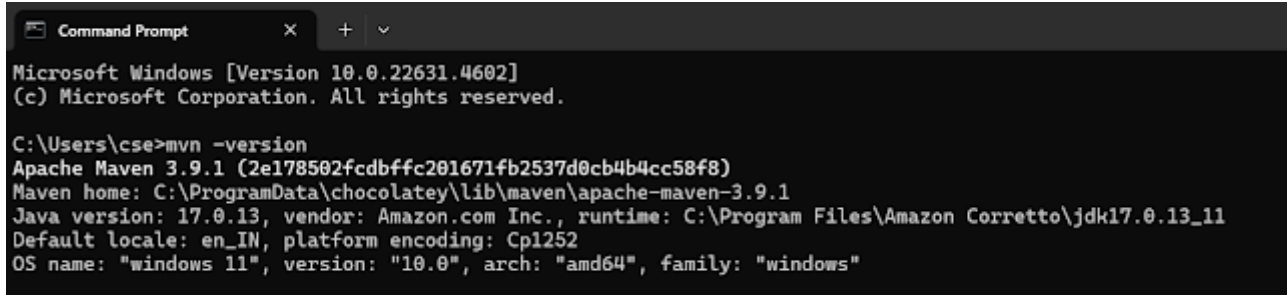
Installation and Setup

Installing Maven on Windows

1. **Download Maven:**
 - a. Go to the [Maven Download Page](#) and download the latest binary ZIP file.
2. **Extract the ZIP File:**
 - a. Right-click the downloaded ZIP file and select **Extract All...** or use any extraction tool like WinRAR or 7-Zip.
3. **Move the Folder:**
 - a. After extraction, move the extracted **Maven folder** (usually named **apache-maven-x.x.x**) to a convenient directory like C:\Program Files\.
4. **Navigate to the bin Folder:**
 - a. Open the **Maven folder**, then navigate to the **bin** folder inside.
 - b. Copy the path from the File Explorer address bar(e.g., **C:\Program Files\apache-maven-x.x.x\bin**).
5. **Set Environment Variables:**
 - a. Open the **Start Menu**, search for **Environment Variables**, and select **Edit the system environment variables**.
 - b. Click **Environment Variables**.
 - c. Under **System Variables**, click **New** and add:
 - **Variable Name:** MAVEN_HOME
 - **Variable Value:** C:\apache-maven
 - d. Under **System Variables**:
 - Find the **path**, double click on it and click **New**.
 - Paste the full path to the bin folder of your Maven directory (e.g., **C:\Program Files\apache-maven-x.x.x\bin**).
6. **Save the Changes:** Click **OK** to close the windows and save your changes.
7. **Verify the Installation:**

Open Command Prompt and run: **mvn -v** or **mvn -version**. If Maven is correctly installed, it will display the version number.

Output



```
Microsoft Windows [Version 10.0.22631.4602]
(c) Microsoft Corporation. All rights reserved.

C:\Users\cse>mvn -version
Apache Maven 3.9.1 (2e178502fcd8bffc201671fb2537d0cb4b4cc58f8)
Maven home: C:\ProgramData\chocolatey\lib\maven\apache-maven-3.9.1
Java version: 17.0.13, vendor: Amazon.com Inc., runtime: C:\Program Files\Amazon Corretto\jdk17.0.13_11
Default locale: en_IN, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

Installing Gradle on Windows

1. Download

Gradle:

Visit the [Gradle Downloads Page](#) and download the latest binary ZIP file.

2. Extract the ZIP File:

- Right-click the downloaded ZIP file and select **Extract All...** or use any extraction tool like WinRAR or 7-Zip.

3. Move the Folder:

- After extraction, move the extracted **Gradle folder** (usually named **gradle-x.x.x**) to a convenient directory like C:\Program Files\.

4. Navigate to the bin Folder:

- Open the **Gradle folder**, then navigate to the **bin** folder inside.
- Copy the path from the File Explorer address bar (e.g., C:\Program Files\gradle-x.x.x\bin).

5. Set Environment Variables:

- a. Open the **Start Menu**, search for **Environment Variables**, and select **Edit the system environment variables**.
- b. Click **Environment Variables**.
- c. Under **System Variables**, click **New** and add:
 - **Variable Name:** GRADLE_HOME
 - **Variable Value:** C:\gradle\gradle-<version>
- d. Under **System Variables**:
 - Find the **path**, double click on it and click **New**.
 - Paste the full path to the bin folder of your Gradle directory (e.g., C:\Program Files\gradle-x.x.x\bin).

6. Save the Changes:

- Click **OK** to close the windows and save your changes.

7. Verify the [Installation](#):

Open a terminal or Command Prompt and run: **gradle -v** or **gradle --version**. If it shows the Gradle version, the setup is complete.

Output

```
C:\Users\cse>gradle -version

-----
Gradle 8.12.1
-----

Build time:    2025-01-24 12:55:12 UTC
Revision:     0b1ee1ff81d1f4a26574ff4a362ac9180852b140

Kotlin:       2.0.21
Groovy:       3.0.22
Ant:          Apache Ant(TM) version 1.10.15 compiled on August 25 2024
Launcher JVM: 17.0.13 (Amazon.com Inc. 17.0.13+11-LTS)
Daemon JVM:   C:\Program Files\Amazon Corretto\jdk17.0.13_11 (no JDK specified, using current Java home)
OS:           Windows 11 10.0 amd64
```


Viva Questions:**1. What is the purpose of build automation tools like Maven and Gradle?****Answer:**

Build automation tools automate the process of compiling, testing, packaging, and deploying software projects. They help reduce manual errors, ensure consistency, and improve productivity in software development.

2. What is the main difference between Maven and Gradle?**Answer:**

The key difference is that **Maven** uses an XML-based configuration (pom.xml), while **Gradle** uses a Groovy/Kotlin-based configuration (build.gradle). Gradle is also faster due to its incremental build mechanism, whereas Maven follows a more rigid, convention-based lifecycle.

3. What is the role of a dependency management system in Maven and Gradle?**Answer:**

Dependency management systems handle external libraries required for a project. Maven uses a pom.xml file to define dependencies, while Gradle uses a build.gradle file. These tools automatically download and manage required dependencies from repositories like **Maven Central**.

4. How does Maven define the build lifecycle?**Answer:**

Maven follows a **predefined build lifecycle** that consists of different phases:

1. **validate** – Checks project configuration.
2. **compile** – Compiles the source code.
3. **test** – Runs unit tests.
4. **package** – Packages the compiled code into a .jar or .war file.
5. **install** – Installs the package locally.
6. **deploy** – Deploys the package to a remote repository.

5. What command is used to check the installed version of Maven and Gradle?

Answer:

- To check Maven version:
- `mvn -version`
- To check Gradle version:
- `gradle -version`

6. What are the advantages of using Gradle over Maven?

Answer:

- **Faster build times** due to **incremental builds** and **caching**.
- **More flexible** configuration using Groovy/Kotlin DSL.
- **Better support for multi-language projects** beyond Java, such as Kotlin, C++, and Python.

7. What is a POM file in Maven?

Answer:

The **Project Object Model (POM)** file (`pom.xml`) is the core configuration file in a Maven project. It defines the project structure, dependencies, plugins, and build settings.

Example of a simple `pom.xml` file:

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>myapp</artifactId>
  <version>1.0.0</version>
</project>
```

8. What is the Gradle equivalent of Maven's POM file?

Answer:

In Gradle, the build.gradle file serves the same purpose as Maven's pom.xml. It is used to define dependencies, plugins, and tasks.

Example of a basic build.gradle file in **Groovy**:

```
plugins {  
    id 'java'  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation 'org.springframework:spring-core:5.3.9'  
}
```

9. How can you install and set up Maven on a system?**Answer:**

1. **Download** Apache Maven from the <https://maven.apache.org/download.cgi>
2. **Extract** the downloaded archive to a suitable location (e.g., C:\apache-maven on Windows).
3. **Set environment variables:**
 - MAVEN_HOME = C:\apache-maven
 - Add %MAVEN_HOME%\bin to the system PATH.
4. **Verify installation** using the command:
5. mvn -version

10. How does Gradle achieve better performance than Maven?**Answer:**

Gradle is faster than Maven because:

- It uses **incremental builds**, meaning it rebuilds only modified parts of the code.
- It employs **caching** to store previously built results and avoid unnecessary recompilation.
- It supports **parallel execution**, utilizing multiple CPU cores to speed up tasks.

Experiment-2

Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins.

What is Maven?

Apache Maven is a **build automation and project management tool** that simplifies software development by managing dependencies, compiling code, running tests, and packaging applications. It follows a **convention-over-configuration** approach and is widely used in **Java-based projects**.

Why Use Maven?

- Standardized project structure.
- **Automatic dependency management** using `pom.xml`.
- Integration with **CI/CD pipelines** (Jenkins, GitHub Actions, Azure DevOps).
- Supports various **build lifecycle phases** (compile, test, package, deploy).

Creating a Maven Project

Step 1: Install Maven (If Not Installed)

Check if Maven is installed using:

```
mvn -version
```

If not installed, follow **Exercise 1** for installation steps.

Step 2: Create a Maven Project Using the Command Line

Open **Command Prompt**:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=myapp  
-DarchetypeArtifactId=maven-archetype-quickstart -  
DinteractiveMode=false
```

Explanation:

- `-DgroupId=com.example` → Defines the package structure (similar to a namespace).
- `-DartifactId=myapp` → The project name and the name of the final artifact (JAR file).
- `-DarchetypeArtifactId=maven-archetype-quickstart` → Uses a pre-defined **template** for Java projects.

- -DinteractiveMode=false → Skips manual inputs and creates the project automatically.

Output:

```
Microsoft Windows [Version 10.0.22631.4602]
(c) Microsoft Corporation. All rights reserved.

C:\Users\cse>mvn archetype:generate -DgroupId=com.example -DartifactId=myapp -DarchetypeArtifactId=maven-archetype-quickstart -Dinter
activeMode=false
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/3.2.0/maven-clean-plugin-3
.2.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/3.2.0/maven-clean-plugin-3.
2.0.pom (5.3 kB at 9.8 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/35/maven-plugins-35.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/35/maven-plugins-35.pom (9.9 kB
at 174 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/35/maven-parent-35.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/35/maven-parent-35.pom (45 kB at 696 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/25/apache-25.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/25/apache-25.pom (21 kB at 622 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/3.2.0/maven-clean-plugin-3
.2.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/3.2.0/maven-clean-plugin-3.
2.0.jar (36 kB at 759 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.0/maven-install-plug
in-3.1.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.0/maven-install-plugi
n-3.1.0.pom (7.8 kB at 124 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/37/maven-plugins-37.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/37/maven-plugins-37.pom (9.9 kB
at 268 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/37/maven-parent-37.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/37/maven-parent-37.pom (46 kB at 971 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/27/apache-27.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/27/apache-27.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-script-interpreter/1.5/maven-script-inter
preter-1.5.jar (25 kB at 13 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache-extras/beanshell/bsh/2.0b6/bsh-2.0b6.jar
Downloading from central: https://repo.maven.apache.org/maven2/commons-collections/commons-collections/3.2.2/commons-collections-3.2.2
.jar (588 kB at 287 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache-extras/beanshell/bsh/2.0b6/bsh-2.0b6.jar (389 kB at 179 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/github/luben/zstd-jni/1.5.6-3/zstd-jni-1.5.6-3.jar (6.7 MB at 2.4 M
B/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/groovy/groovy/4.0.23/groovy-4.0.23.jar (7.6 MB at 1.5 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/ibm/icu/icu4j/75.1/icu4j-75.1.jar (14 MB at 2.6 MB/s)
[INFO] Generating project in Batch mode
Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (16 MB at 5.5 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-quickstart/1.0/maven-archet
ype-quickstart-1.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-quickstart/1.0/maven-archet
ype-quickstart-1.0.jar (4.3 kB at 90 kB/s)
[INFO] -----
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] -----
[INFO] Parameter: basedir, Value: C:\Users\cse
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: myapp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: C:\Users\cse\myapp
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

Step 3: Navigate to the Project Folder

cd myapp

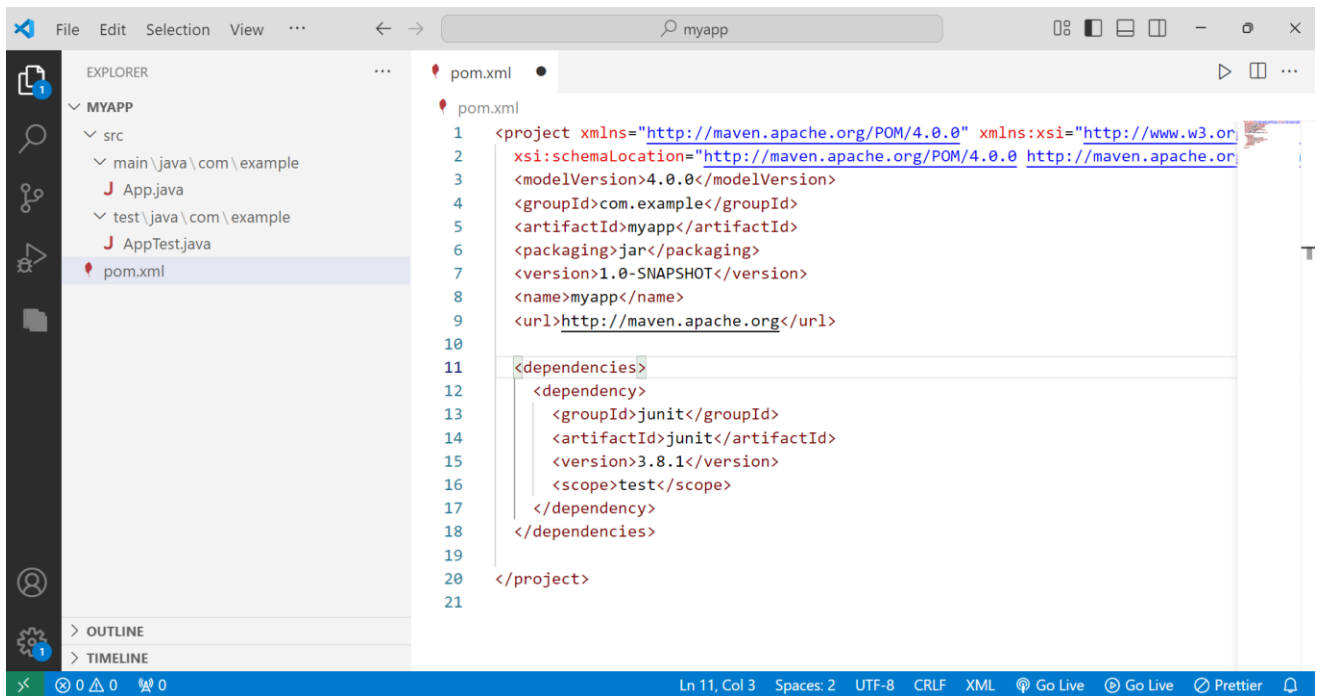
Step 4: Verify the Project Structure

Maven creates a standard folder structure like this:

```
myapp
├── src
│   ├── main
│   │   └── java
│   │       └── com.example
│   │           └── App.java
│   └── test
│       └── java
│           └── com.example
│               └── AppTest.java
├── pom.xml
├── README.md
└── .gitignore
```

- **src/main/java** → Contains the main application code.
- **src/test/java** → Contains test cases for the application.
- **pom.xml** → The **core configuration file** for Maven.

Output:



```
1 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
3   <modelVersion>4.0.0</modelVersion>
4   <groupId>com.example</groupId>
5   <artifactId>myapp</artifactId>
6   <packaging>jar</packaging>
7   <version>1.0-SNAPSHOT</version>
8   <name>myapp</name>
9   <url>http://maven.apache.org</url>
10
11   <dependencies>
12     <dependency>
13       <groupId>junit</groupId>
14       <artifactId>junit</artifactId>
15       <version>3.8.1</version>
16       <scope>test</scope>
17     </dependency>
18   </dependencies>
19
20 </project>
21
```

Note: Used VS Code Text Editor. Make 'myapp' as your project folder

Understanding the POM File (pom.xml)

The **Project Object Model (POM)** file is the heart of a Maven project. It defines **project metadata, dependencies, plugins, and build configurations**.

Example pom.xml File

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>myapp</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <!-- Example Dependency: JUnit for Testing -->
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- Maven Compiler Plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>
        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Key Sections of pom.xml

1. **<groupId>** - Unique identifier for the project (e.g., com.example).
2. **<artifactId>** - Project name (e.g., myapp).
3. **<version>** - Version of the project (e.g., 1.0-SNAPSHOT).
4. **<dependencies>** - Lists external libraries required for the project.
5. **<build>** - Specifies plugins for compiling, packaging, and testing.

Managing Dependencies in Maven

Maven automatically downloads required libraries from the **Maven Central Repository**.

Adding a Dependency

To use **Spring Boot**, add the following inside the `<dependencies>` section:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>2.7.0</version>
</dependency>
```

After adding dependencies, run:

```
mvn clean install
```

This downloads and installs required JAR files.

Output:

```

C:\Users\cse\myapp>mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:myapp >-----
[INFO] Building myapp 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/3.3.0/maven-resources-
plugin-3.3.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/3.3.0/maven-resources-p
ugin-3.3.0.pom (8.5 kB at 10 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/3.3.0/maven-resources-
plugin-3.3.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/3.3.0/maven-resources-p
ugin-3.3.0.jar (32 kB at 216 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.10.1/maven-compiler-p
ugin-3.10.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.10.1/maven-compiler-pl
ugin-3.10.1.pom (13 kB at 268 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/34/maven-plugins-34.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/34/maven-plugins-34.pom (11 kB a
t 223 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/34/maven-parent-34.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/34/maven-parent-34.pom (43 kB at 1.3 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/tomcat/embed/tomcat-embed-core/9.0.63/tomcat-embed-core-9.0.
63.jar (3.4 MB at 1.4 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/spring-context/5.3.20/spring-context-5.3.20.jar (1.
3 MB at 428 kB/s)
[INFO] --- clean:3.2.0:clean (default-clean) @ myapp ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.3.4/maven-shared-utils-3.
3.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.3.4/maven-shared-utils-3.3
.4.pom (5.8 kB at 124 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-components/34/maven-shared-compon
ents-34.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-components/34/maven-shared-compon
ents-34.pom
Downloaded from central: https://repo.maven.apache.org/maven2/commons-io/commons-io/2.6/commons-io-2.6.jar (215 kB at 1.1 MB/s)
[INFO] --- resources:3.3.0:resources (default-resources) @ myapp ---
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-interpolation/1.26/plexus-interpolation-1.2
6.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-interpolation/1.26/plexus-interpolation-1.26
.pom (2.7 kB at 34 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/5.1/plexus-5.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/5.1/plexus-5.1.pom (23 kB at 363 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-filtering/3.3.0/maven-filtering-3.3.0.po
m
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-filtering/3.3.0/maven-filtering-3.3.0.pom
(6.9 kB at 124 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-lang3/3.12.0/commons-lang3-3.12.0.jar (587 k
B at 2.3 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/commons-io/commons-io/2.11.0/commons-io-2.11.0.jar (327 kB at 1.2 MB/s)
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\cse\myapp\src\main\resources
[INFO] --- compiler:3.10.1:compile (default-compile) @ myapp ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-incremental/1.1/maven-shared-incr
emental-1.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-incremental/1.1/maven-shared-incr
emental-1.1.pom (4.7 kB at 176 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/thoughtworks/qdox/qdox/2.0.1/qdox-2.0.1.jar (334 kB at 2.3 MB/s)
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file to C:\Users\cse\myapp\target\classes
[INFO] --- resources:3.3.0:testResources (default-testResources) @ myapp ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\cse\myapp\src\test\resources
[INFO] --- compiler:3.10.1:testCompile (default-testCompile) @ myapp ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file to C:\Users\cse\myapp\target\test-classes
[INFO] --- surefire:3.0.0:test (default-test) @ myapp ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/maven-surefire-common/3.0.0/maven-surefire-c
ommon-3.0.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.0.0/common-java5-3.0.0.jar (18
kB at 399 kB/s)
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ myapp ---
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/file-management/3.1.0/file-management-3.1.0.po
m
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/file-management/3.1.0/file-management-3.1.0.pom
(4.5 kB at 95 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.21/commons-compress-1.21.jar (1.0
MB at 3.1 MB/s)
[INFO] Building jar: C:\Users\cse\myapp\target\myapp-1.0-SNAPSHOT.jar
[INFO] --- install:3.1.0:install (default-install) @ myapp ---
[INFO] Installing C:\Users\cse\myapp\pom.xml to C:\Users\cse\.m2\repository\com\example\myapp\1.0-SNAPSHOT\myapp-1.0-SNAPSHOT.pom
[INFO] Installing C:\Users\cse\myapp\target\myapp-1.0-SNAPSHOT.jar to C:\Users\cse\.m2\repository\com\example\myapp\1.0-SNAPSHOT\myap
p-1.0-SNAPSHOT.jar
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 24.451 s
[INFO] Finished at: 2025-02-06T10:26:00+05:30
[INFO] -----

```

- All downloaded dependencies are stored in `.m2/repository`.

Output:

```
C:\Users\cse\.m2\repository\junit\junit\3.8.1>dir
Volume in drive C is Windows
Volume Serial Number is AAC5-EAC8

Directory of C:\Users\cse\.m2\repository\junit\junit\3.8.1

06-02-2025  10:25    <DIR>          .
06-02-2025  10:41    <DIR>          ..
06-02-2025  10:25             121,070 junit-3.8.1.jar
06-02-2025  10:25              40 junit-3.8.1.jar.sha1
06-02-2025  10:25             998 junit-3.8.1.pom
06-02-2025  10:25              40 junit-3.8.1.pom.sha1
06-02-2025  10:25             194 _remote.repositories
                    5 File(s)          122,342 bytes
                    2 Dir(s)  317,208,489,984 bytes free

C:\Users\cse\.m2\repository\junit\junit\3.8.1>
```

- Use `mvn dependency:tree` to list all dependencies.

Output:

```

C:\Users\cse\myapp>mvn dependency:tree
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:myapp >-----
[INFO] Building myapp 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- dependency:2.8:tree (default-cli) @ myapp ---
[WARNING] Parameter 'localRepository' is deprecated core expression; Avoid use of ArtifactRepository type. If you need access to local repository, switch to '${repositorySystemSession}' expression and get LRM from it instead.
[INFO] com.example:myapp:jar:1.0-SNAPSHOT
[INFO] +- junit:junit:jar:3.8.1:test
[INFO] \- org.springframework.boot:spring-boot-starter-web:jar:2.7.0:compile
[INFO]     +- org.springframework.boot:spring-boot-starter:jar:2.7.0:compile
[INFO]     | +- org.springframework.boot:spring-boot:jar:2.7.0:compile
[INFO]     | +- org.springframework.boot:spring-boot-autoconfigure:jar:2.7.0:compile
[INFO]     | +- org.springframework.boot:spring-boot-starter-logging:jar:2.7.0:compile
[INFO]     | | +- ch.qos.logback:logback-classic:jar:1.2.11:compile
[INFO]     | | +- ch.qos.logback:logback-core:jar:1.2.11:compile
[INFO]     | | \- org.slf4j:slf4j-api:jar:1.7.32:compile
[INFO]     | +- org.apache.logging.log4j:log4j-to-slf4j:jar:2.17.2:compile
[INFO]     | | \- org.apache.logging.log4j:log4j-api:jar:2.17.2:compile
[INFO]     | \- org.slf4j:jul-to-slf4j:jar:1.7.36:compile
[INFO]     +- jakarta.annotation:jakarta.annotation-api:jar:1.3.5:compile
[INFO]     +- org.springframework:spring-core:jar:5.3.20:compile
[INFO]     | \- org.springframework:spring-jcl:jar:5.3.20:compile
[INFO]     \- org.yaml:snakeyaml:jar:1.30:compile
[INFO] +- org.springframework.boot:spring-boot-starter-json:jar:2.7.0:compile
[INFO] | +- com.fasterxml.jackson.core:jackson-databind:jar:2.13.3:compile
[INFO] | | +- com.fasterxml.jackson.core:jackson-annotations:jar:2.13.3:compile
[INFO] | | \- com.fasterxml.jackson.core:jackson-core:jar:2.13.3:compile
[INFO] | +- jakarta.annotation:jakarta.annotation-api:jar:1.3.5:compile
[INFO] | +- org.springframework:spring-core:jar:5.3.20:compile
[INFO] | | \- org.springframework:spring-jcl:jar:5.3.20:compile
[INFO] | \- org.yaml:snakeyaml:jar:1.30:compile
[INFO] +- org.springframework.boot:spring-boot-starter-json:jar:2.7.0:compile
[INFO] | +- com.fasterxml.jackson.core:jackson-databind:jar:2.13.3:compile
[INFO] | | +- com.fasterxml.jackson.core:jackson-annotations:jar:2.13.3:compile
[INFO] | | \- com.fasterxml.jackson.core:jackson-core:jar:2.13.3:compile
[INFO] | +- com.fasterxml.jackson.datatype:jackson-datatype-jdk8:jar:2.13.3:compile
[INFO] | +- com.fasterxml.jackson.datatype:jackson-datatype-jsr310:jar:2.13.3:compile
[INFO] | \- com.fasterxml.jackson.module:jackson-module-parameter-names:jar:2.13.3:compile
[INFO] +- org.springframework.boot:spring-boot-starter-tomcat:jar:2.7.0:compile
[INFO] | +- org.apache.tomcat.embed:tomcat-embed-core:jar:9.0.63:compile
[INFO] | +- org.apache.tomcat.embed:tomcat-embed-el:jar:9.0.63:compile
[INFO] | \- org.apache.tomcat.embed:tomcat-embed-websocket:jar:9.0.63:compile
[INFO] +- org.springframework:spring-web:jar:5.3.20:compile
[INFO] | \- org.springframework:spring-beans:jar:5.3.20:compile
[INFO] \- org.springframework:spring-webmvc:jar:5.3.20:compile
[INFO]     +- org.springframework:spring-aop:jar:5.3.20:compile
[INFO]     +- org.springframework:spring-context:jar:5.3.20:compile
[INFO]     \- org.springframework:spring-expression:jar:5.3.20:compile
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.458 s
[INFO] Finished at: 2025-02-06T10:51:41+05:30
[INFO]

```

- You can open `.jar` files using an archive tool or the `jar` command.

Output:

```
C:\Users\cse\.m2\repository\junit\junit\3.8.1>jar tf junit-3.8.1.jar
META-INF/
META-INF/MANIFEST.MF
junit/
junit/awtui/
junit/extensions/
junit/framework/
junit/runner/
junit/swingui/
junit/swingui/icons/
junit/textui/
junit3.8.1/
junit/awtui/AboutDialog$1.class
junit/awtui/AboutDialog$2.class
junit/awtui/AboutDialog.class
junit/awtui/Logo.class
junit/awtui/ProgressBar.class
junit/awtui/TestRunner$1.class
junit/awtui/TestRunner$10.class
junit/awtui/TestRunner$2.class
junit/awtui/TestRunner$3.class
junit/awtui/TestRunner$4.class
junit/awtui/TestRunner$5.class
junit/awtui/TestRunner$6.class
junit/awtui/TestRunner$7.class
junit/awtui/TestRunner$8.class
junit/awtui/TestRunner$9.class
junit/awtui/TestRunner.class
junit/extensions/ActiveTestSuite$1.class
junit/extensions/ActiveTestSuite.class
junit/extensions/ExceptionTestCase.class
```

Using Maven Plugins

Maven **plugins** automate tasks like compiling code, running tests, and packaging the application.

Common Maven Plugins

Plugin	Purpose
maven-compiler-plugin	Compiles Java code
maven-surefire-plugin	Runs unit tests
maven-jar-plugin	Packages the project as a JAR file

Practically Using Maven Plugins in a Project

To compile code, run unit tests, and package the application using Maven plugins, follow this step-by-step guide.

Step1: Create Maven Project (use the above procedure)

Step2: Configuring Maven Plugins in `pom.xml`

```
<build>
```



```
<plugins>
  <!-- Compiler Plugin to compile Java code -->
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.8.1</version>
    <configuration>
      <source>1.8</source>
      <target>1.8</target>
    </configuration>
  </plugin>

  <!-- Surefire Plugin to run unit tests -->
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.0.0-M7</version>
  </plugin>

  <!-- JAR Plugin to package the application -->
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-jar-plugin</artifactId>
    <version>3.2.0</version>
  </plugin>
</plugins>
</build>
```

This configures Maven to:

- ✔ Compile Java code using the `maven-compiler-plugin`.
- ✔ Run unit tests using the `maven-surefire-plugin`.
- ✔ Package the application into a JAR file using the `maven-jar-plugin`.

Step3: Writing a Sample Java Program (App.java)

```
package com.example;

public class App {
    public static String greet() {
        return "Hello, DevOps!";
    }

    public static void main(String[] args) {
        System.out.println(greet());
    }
}
```

```
}
```

Step4: Writing a Unit Test (AppTest.java)

Modify the AppTest.java file inside src/test/java/com/example/ to test the greet method:

```
package com.example;

import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class AppTest {
    @Test
    public void testGreet() {
        assertEquals("Hello, DevOps!", App.greet());
    }
}
```

Here:

✓ We use **JUnit** to test the greet() method.

✓ The assertEquals() method checks if the output matches "Hello, DevOps!".

Step5: Running Maven Commands

a. mvn compile (compile the code)

If successful, you will see

```
[INFO] Compiling 1 source file to target/classes
```

b. mvn test (run unit tests)

Expected output (if the test passes)

```
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0,
Time elapsed: 0.01 sec
[INFO] BUILD SUCCESS
```

If the test fails, check the assertion inside AppTest.java.

c. mvn package (package the application)

This will generate the output

```
[INFO] Building jar: /path-to-project/myapp/target/myapp-1.0-SNAPSHOT.jar
```

The JAR file will be inside the `target/` folder.

d. `java -jar target/myapp-1.0-SNAPSHOT.jar` (run the jar file)

Output

Hello, DevOps!

Experiment-3

Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation.

What is Gradle?

Gradle is a **modern, flexible, and high-performance** build automation tool used in Java, Kotlin, and Android projects. It is designed to **optimize build performance** using **incremental builds** and **task caching**.

Why Use Gradle?

- **Faster than Maven** due to incremental builds.
- **Uses Groovy/Kotlin DSL** instead of XML.
- **Highly customizable** build automation.
- **Supports dependency management** like Maven.
- **Works with multiple languages** (Java, Kotlin, C++, Python, etc.).

Setting Up a Gradle Project

Step 1: Install Gradle (If Not Installed)

Check if Gradle is installed using:

```
gradle -version
```

If not installed, follow **Exercise 1** for installation steps.

Step 2: Create a New Gradle Project

To create a **Java-based Gradle project**, run:

```
mkdir myapp  
cd myapp  
gradle init
```

Follow the prompts:

- **Select Project Type:** Choose application.
- **Select Build Script Language:** Choose Groovy or Kotlin.
- **Select Java Version:** Choose 8, 11, 17, or latest.

Output:

```

C:\Users\cse>gradle init
Starting a Gradle Daemon (subsequent builds will be faster)

Found existing files in the project directory: 'C:\Users\cse'.
Directory will be modified and existing files may be overwritten. Continue? (default: no) [yes, no] yes

Select type of build to generate:
 1: Application
 2: Library
 3: Gradle plugin
 4: Basic (build structure only)
Enter selection (default: Application) [1..4] 1

Select implementation language:
 1: Java
 2: Kotlin
 3: Groovy
 4: Scala
 5: C++
 6: Swift
Enter selection (default: Java) [1..6] 3

Enter target Java version (min: 7, default: 21): 8

Project name (default: cse): myapp

Select application structure:
 1: Single application project
 2: Application and library project
Enter selection (default: Single application project) [1..2] 1

Select build script DSL:
 1: Kotlin
 2: Groovy
Enter selection (default: Groovy) [1..2] 2

Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] yes

> Task :init
Learn more about Gradle by exploring our Samples at https://docs.gradle.org/8.12.1/samples/sample_building_groovy_applications.html

BUILD SUCCESSFUL in 1m 39s
1 actionable task: 1 executed
C:\Users\cse>

```

Once the setup is complete, navigate to the project folder:

Step 3: Verify the Project Structure

Gradle creates a structured project like this:

```

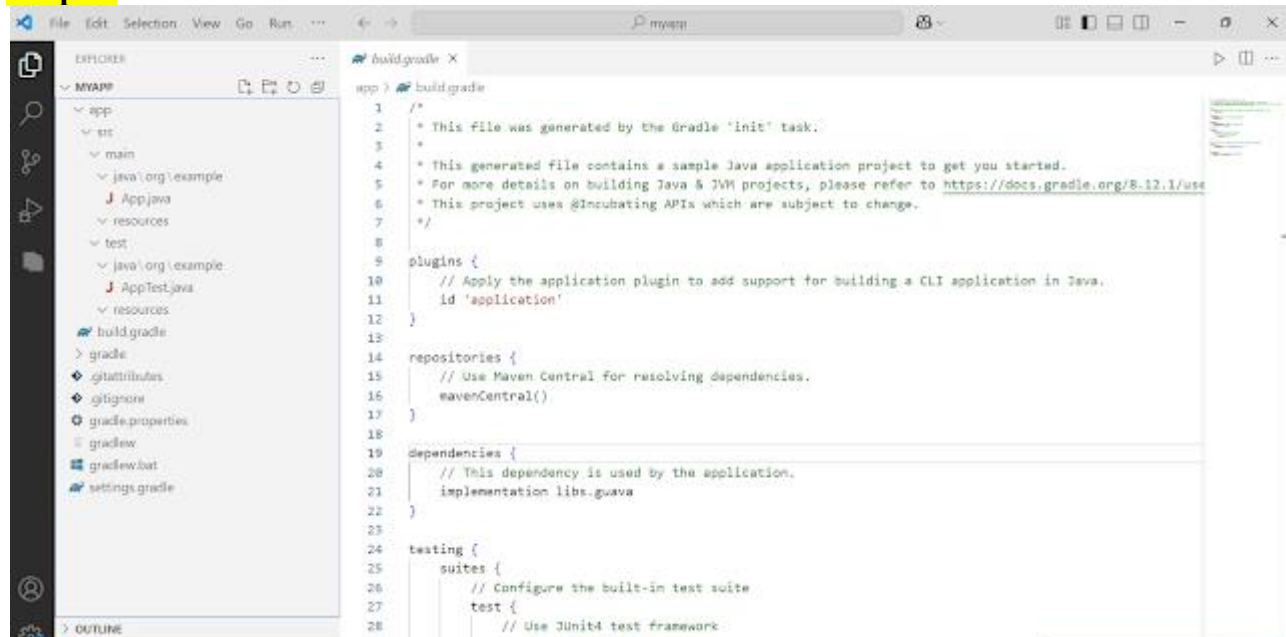
myapp
├── src
│   ├── main
│   │   └── java
│   │       └── com.example
│   │           └── App.java
│   └── test
│       └── java
│           └── com.example
│               └── AppTest.java
├── build.gradle (or build.gradle.kts)
├── settings.gradle
├── gradlew
└── gradlew.bat

```

|— gradle/

- **src/main/java** → Contains main application code.
- **src/test/java** → Contains test cases.
- **build.gradle** or **build.gradle.kts** → The **Gradle build script** (Groovy or Kotlin).

Output:



Understanding Gradle Build Scripts

Gradle supports two script languages:

1. **Groovy DSL** (`build.gradle`) – Default scripting language.
2. **Kotlin DSL** (`build.gradle.kts`) – Modern, type-safe alternative.

Groovy DSL (`build.gradle`) Example

```
plugins {  
    id 'java'  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {
```

```
        implementation 'org.springframework.boot:spring-boot-
starter-web:2.7.0'
        testImplementation 'junit:junit:4.13.2'
    }

    tasks.register('hello') {
        doLast {
            println 'Hello, DevOps with Gradle!'
        }
    }
}
```

Kotlin DSL (build.gradle.kts) Example

```
plugins {
    java
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.springframework.boot:spring-boot-
starter-web:2.7.0")
    testImplementation("junit:junit:4.13.2")
}

tasks.register("hello") {
    doLast {
        println("Hello, DevOps with Gradle!")
    }
}
```

Key Sections in the Build Script

- **plugins** → Specifies project type (e.g., java).
- **repositories** → Defines where dependencies are downloaded from.
- **dependencies** → Lists required libraries.
- **tasks** → Automates build tasks.

Managing Dependencies in Gradle

Gradle uses dependency management similar to Maven.

Adding Dependencies

Inside `build.gradle` or `build.gradle.kts`:

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-  
starter-web:2.7.0'  
    testImplementation 'junit:junit:4.13.2'  
}
```

After adding dependencies, **refresh Gradle** using:

```
gradle build
```

This downloads and caches required JAR files.

Automating Tasks in Gradle

Gradle tasks **automate** common actions like compiling code, running tests, and packaging the project.

Common Gradle Tasks

Command	Purpose
<code>gradle build</code>	Builds the project
<code>gradle test</code>	Runs unit tests
<code>gradle run</code>	Runs the application (if an application plugin is used)
<code>gradle clean</code>	Deletes previous build files
<code>gradle dependencies</code>	Lists all project dependencies
<code>gradle hello</code>	Runs a custom task (from <code>build.gradle</code>)

Running a Custom Task

```
gradle hello
```

Expected Output:

```
Hello, DevOps with Gradle!
```

Running and Packaging the Gradle Project

Step 1: Compile the Code

```
gradle compileJava
```

This compiles the source files in `src/main/java`.

Step 2: Run Unit Tests

```
gradle test
```

If all tests pass, you'll see:

```
BUILD SUCCESSFUL
```

Step 3: Package the Project into a JAR

```
gradle jar
```

This creates a `.jar` file inside the `build/libs/` directory.

Step 4: Run the Application

```
java -jar build/libs/myapp.jar
```

Expected Output:

```
Hello, DevOps with Gradle!
```

Experiment-4

Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle.

Part 1: Build and Run a Java Application Using Maven

Step 1: Create a Maven Project

Open a terminal and run:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=myapp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Then, navigate into the project folder:

```
cd myapp
```

OUTPUT:

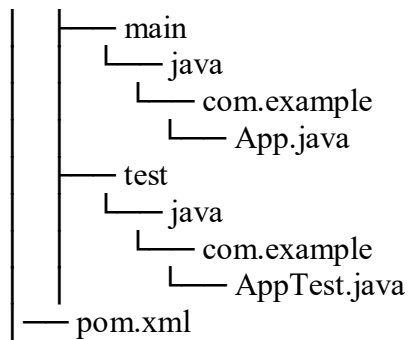
```
Microsoft Windows [Version 10.0.22631.4751]
(c) Microsoft Corporation. All rights reserved.

C:\Users\cse>mvn archetype:generate -DgroupId=com.example -DartifactId=myapp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> archetype:3.3.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< archetype:3.3.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- archetype:3.3.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: C:\Users\cse
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: myapp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: C:\Users\cse\myapp
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.234 s
[INFO] Finished at: 2025-03-07T10:29:52+05:30
```

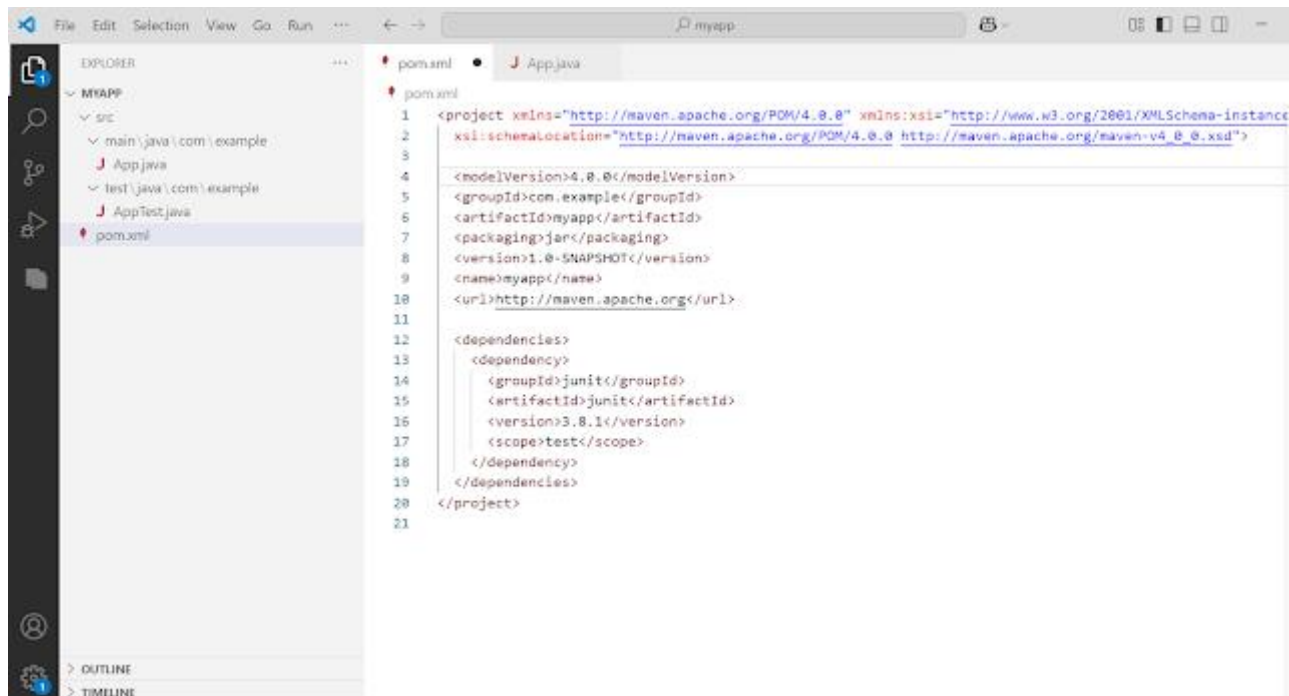
Step 2: Verify Project Structure

After running the command, your project will have this structure:

```
myapp
|—— src
```



OUTPUT:



Step 3: Update App.java (Main Application)

Modify src/main/java/com/example/App.java:

package com.example;

public class App {

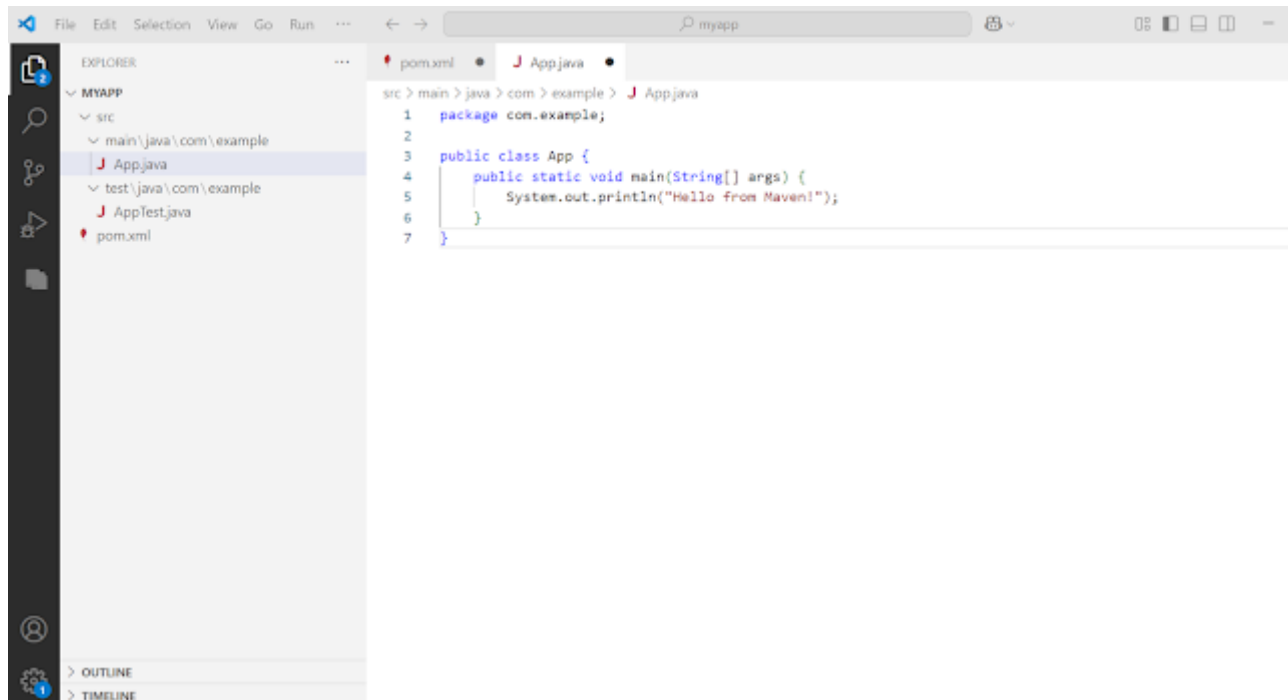
public static void main(String[] args) {

System.out.println("Hello from Maven!");

}

}

OUTPUT:



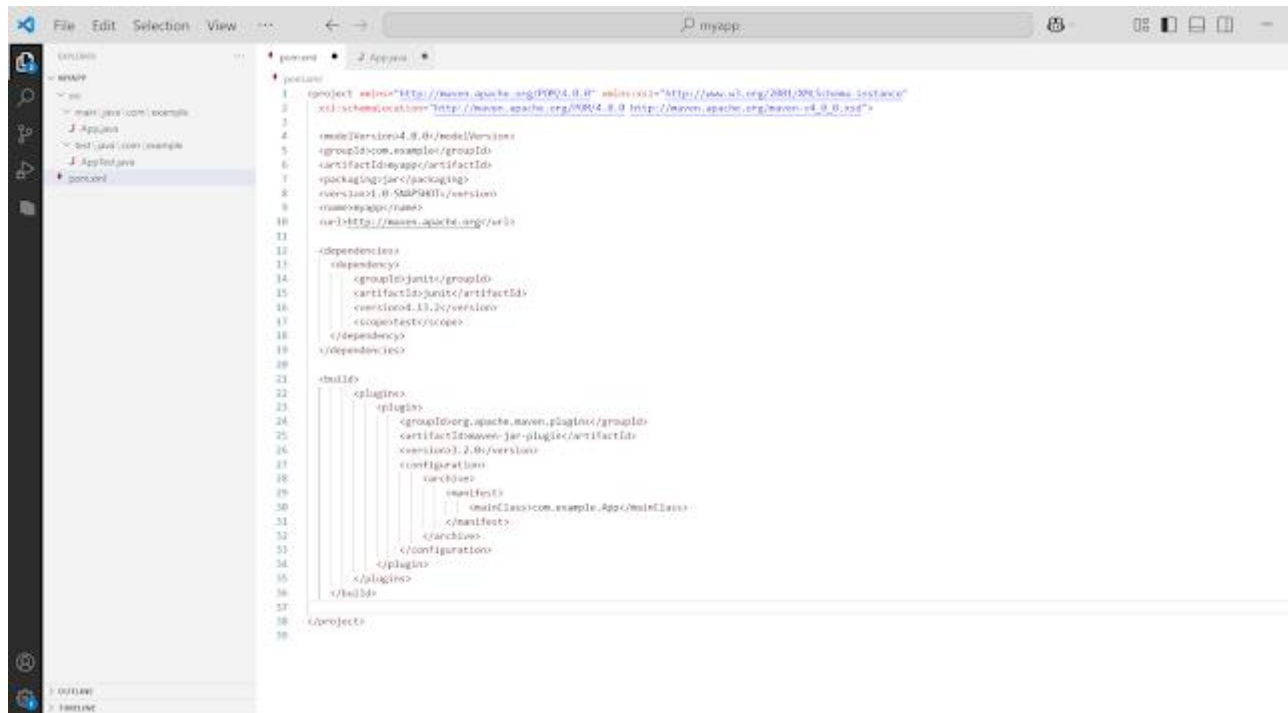
Step 4: Configure pom.xml for JUnit and Plugins

Modify pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>myapp</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <dependencies>
    <dependency>
```

```
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.13.2</version>
<scope>test</scope>
</dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <version>3.2.0</version>
      <configuration>
        <archive>
          <manifest>
            <mainClass>com.example.App</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

OUTPUT:



Step 5: Build and Run the Application with Maven

Compile and Build the JAR

`mvn clean package`

OUTPUT:

```
C:\Users\cse\myapp>mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:myapp >-----
[INFO] Building myapp 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ myapp ---
[INFO]
[INFO] --- resources:3.3.0:resources (default-resources) @ myapp ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\cse\myapp\src\main\resources
[INFO]
[INFO] --- compiler:3.10.1:compile (default-compile) @ myapp ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file to C:\Users\cse\myapp\target\classes
[INFO]
[INFO] --- resources:3.3.0:testResources (default-testResources) @ myapp ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\cse\myapp\src\test\resources
[INFO]
[INFO] --- compiler:3.10.1:testCompile (default-testCompile) @ myapp ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file to C:\Users\cse\myapp\target\test-classes
[INFO]
[INFO] --- surefire:3.0.0:test (default-test) @ myapp ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -----
```

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.032 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.2.0:jar (default-jar) @ myapp ---
[INFO] Building jar: C:\Users\cse\myapp\target\myapp-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 1.983 s
[INFO] Finished at: 2025-03-07T10:37:32+05:30
[INFO] -----
```

Run the Application

```
java -jar target/myapp-1.0-SNAPSHOT.jar
```

Expected Output:

Hello from Maven!

OUTPUT:

```
C:\Users\cse\myapp>dir
Volume in drive C is Windows
Volume Serial Number is AAC5-EAC8

Directory of C:\Users\cse\myapp

07-03-2025  10:37    <DIR>          .
07-03-2025  10:29    <DIR>          ..
07-03-2025  10:36             1,219 pom.xml
07-03-2025  10:29    <DIR>          src
07-03-2025  10:37    <DIR>          target
                1 File(s)              1,219 bytes
                4 Dir(s)  299,416,006,656 bytes free

C:\Users\cse\myapp>java -jar target/myapp-1.0-SNAPSHOT.jar
Hello from Maven!

C:\Users\cse\myapp>
```

Part 2: Migrate the Same Application to Gradle

Step 1: Go to target folder where you can find myapp-1.0-SNAPSHOT.jar file

Navigate to your Maven project target folder and run

```
gradle init
```

Step 2: You can find gradle files created in the same folder (target). Find build.gradle.kts file in app folder. Modify build.gradle.kts (Kotlin DSL)

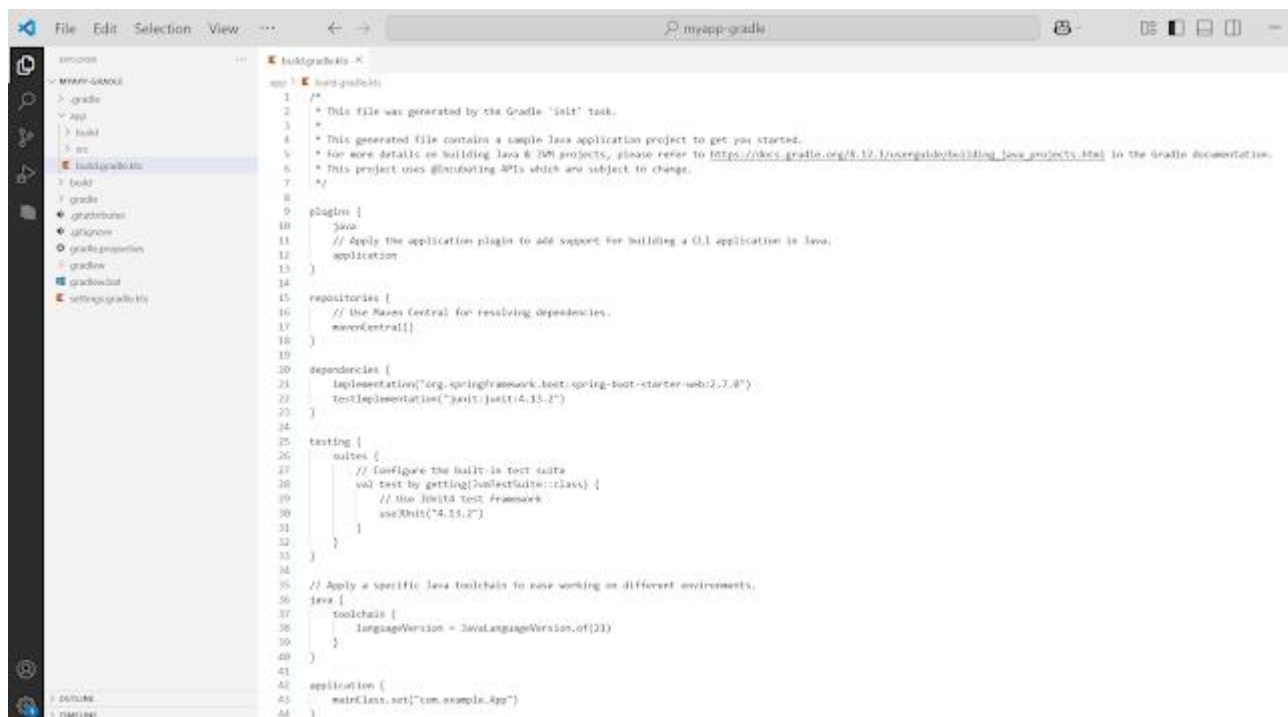
Edit build.gradle.kts:

```
plugins {
    java
    application
}

repositories {
    mavenCentral()
}
```

```
dependencies {  
    implementation("org.springframework.boot:spring-boot-starter-web:2.7.0")  
    testImplementation("junit:junit:4.13.2")  
}  
  
application {  
    mainClass.set("com.example.App")  
}
```

OUTPUT:



Step 3: Build and Run the Application with Gradle

Compile and Build the JAR

gradle clean build

OUTPUT:

```
C:\Users\cse\myapp-gradle>gradle clean build
Reusing configuration cache.

BUILD SUCCESSFUL in 1s
8 actionable tasks: 5 executed, 3 from cache
Configuration cache entry reused.
C:\Users\cse\myapp-gradle>
```

Run the Application

gradle run

Expected Output:

Hello from Maven!

OUTPUT:

```
C:\Users\cse\myapp-gradle>gradle run
Reusing configuration cache.

> Task :app:run
Hello from Maven!

BUILD SUCCESSFUL in 877ms
2 actionable tasks: 1 executed, 1 up-to-date
Configuration cache entry reused.
C:\Users\cse\myapp-gradle>
```



git



GitHub

Introduction to Git & GitHub

- Introduction
- Version Control
 - About Version Control
 - Local Version Control Systems
 - Centralized Version Control Systems
 - Distributed Version Control Systems
- Setting Up Git on Local System
 - Step 1 : Check if git is already installed
 - Step 2 : If not installed download Git installer from <https://git-scm.com/>
 - Step 3 : Run installer and install git
 - Step 4 : Check if git is installed
- Create GitHub Account
 - Step 1 : Go to <https://github.com/> and sign up for a new account
 - Step 2 : Login to GitHub
 - Step 3 : Create a new Repository (Private or Public)
- Basic Git Commands
 - Step 1 : Create a new folder and open Git Bash/Cmd and go to the folder location
 - Step 2 : Run these commands for git configuration
 - Step 3 : Initialize git using this command
 - Step 4 : Add some sample files in the folder by using this command
 - Step 5 : Run these commands to check status, add files and commit your changes/updates
- Git Branches
- Steps to follow while working with branches in Git
 - Step 1 : Create branch `git branch "branch_name"`
 - Step 2 : Checkout branch `git checkout "branch_name"`
 - Step 3 : Make some changes to your project
 - Step 4 : On local repo checkout to master branch `"git checkout master"`
 - Step 5 : Merge new branch in master branch `"git merge branch_name"`
 - Step 6 : Push all your changes `"git push -u origin master"`
 - Step 7 : Delete a particular branch
- Git Tags
 - Step 1 : Checkout the branch where you want to create the tag
 - Step 2 : Create a tag with some name
 - Step 3 : Display or Show tags
 - Step 3 : Push tags to remote repository `git push origin v1.0`
 - Step 4 : Delete tags (Only if required)
- Checking out with tags
- Creating tags from past commits

- Git Merge Conflicts
 - Understanding merge conflicts
 - Types of merge conflicts
 - Git fails to start the merge
 - Git fails during the merge
 - How to identify merge conflicts
 - How to Resolve Merge Conflicts in Git?
 - Git Commands to Resolve Conflicts
 - 1. git log --merge
 - 2. git diff
 - 3. git checkout
 - 4. git reset --mixed
 - 5. git merge --abort
 - 6. git reset



Git & GitHub

Introduction

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

Version Control

About Version Control

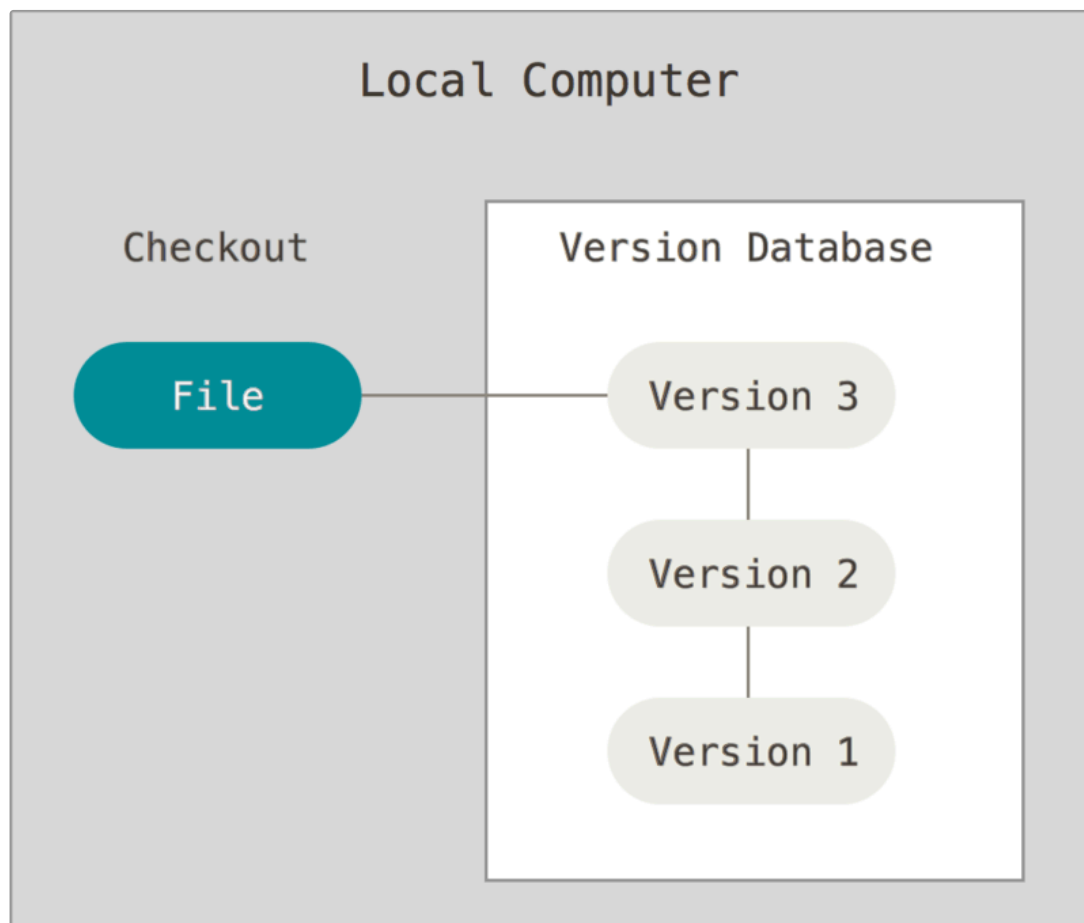
What is “version control”, and why should you care? Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later. For the examples in this book, you will use software source code as the files being version controlled, though in reality you can do this with nearly any type of file on a computer.

If you are a graphic or web designer and want to keep every version of an image or layout (which you would most certainly want to), a Version Control System (VCS) is a very wise thing to use. It allows you to revert selected files back to a previous state, revert the entire project back to a previous state, compare changes over time, see who last modified something that might be causing a problem, who introduced an issue and when, and more. Using a VCS also generally means that if you screw things up or lose files, you can easily recover. In addition, you get all this for very little overhead.

Local Version Control Systems

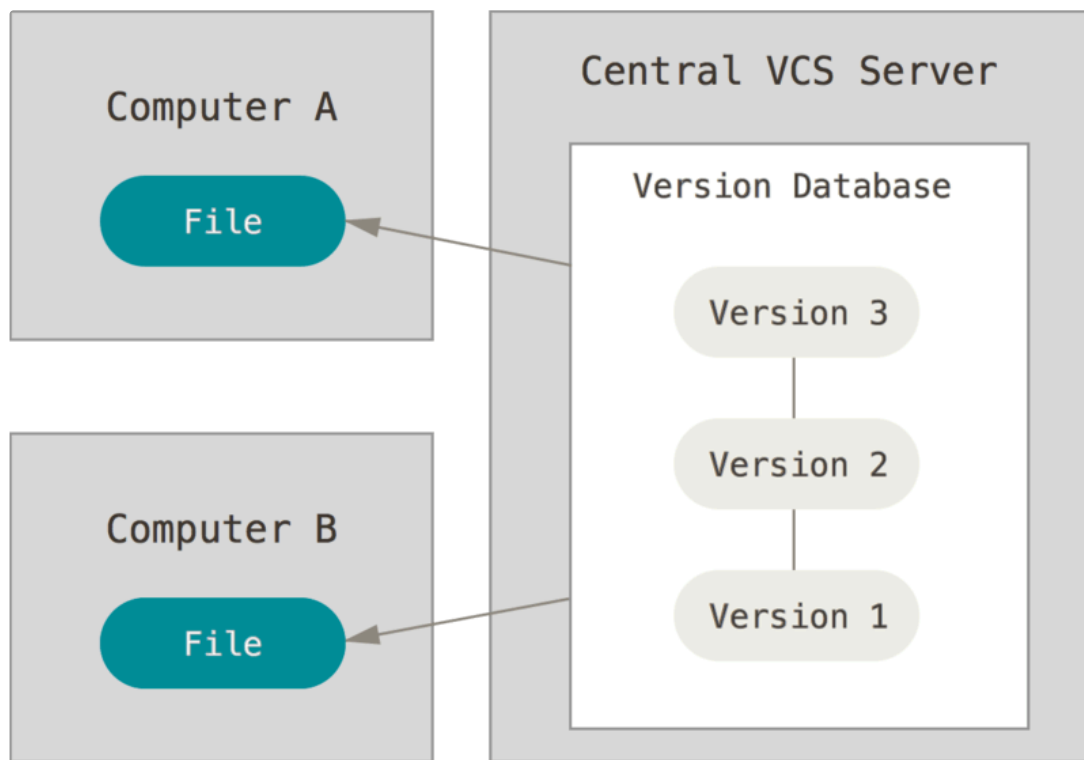
Many people’s version-control method of choice is to copy files into another directory (perhaps a time-stamped directory, if they’re clever). This approach is very common because it is so simple, but it is also incredibly error prone. It is easy to forget which directory you’re in and accidentally write to the wrong file or copy over files you don’t mean to.

To deal with this issue, programmers long ago developed local VCSs that had a simple database that kept all the changes to files under revision control.



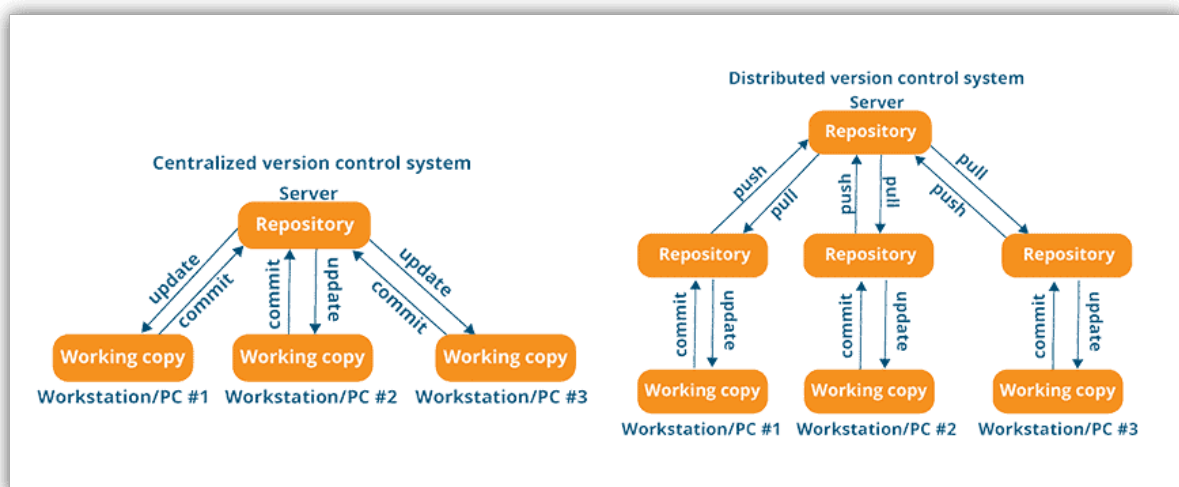
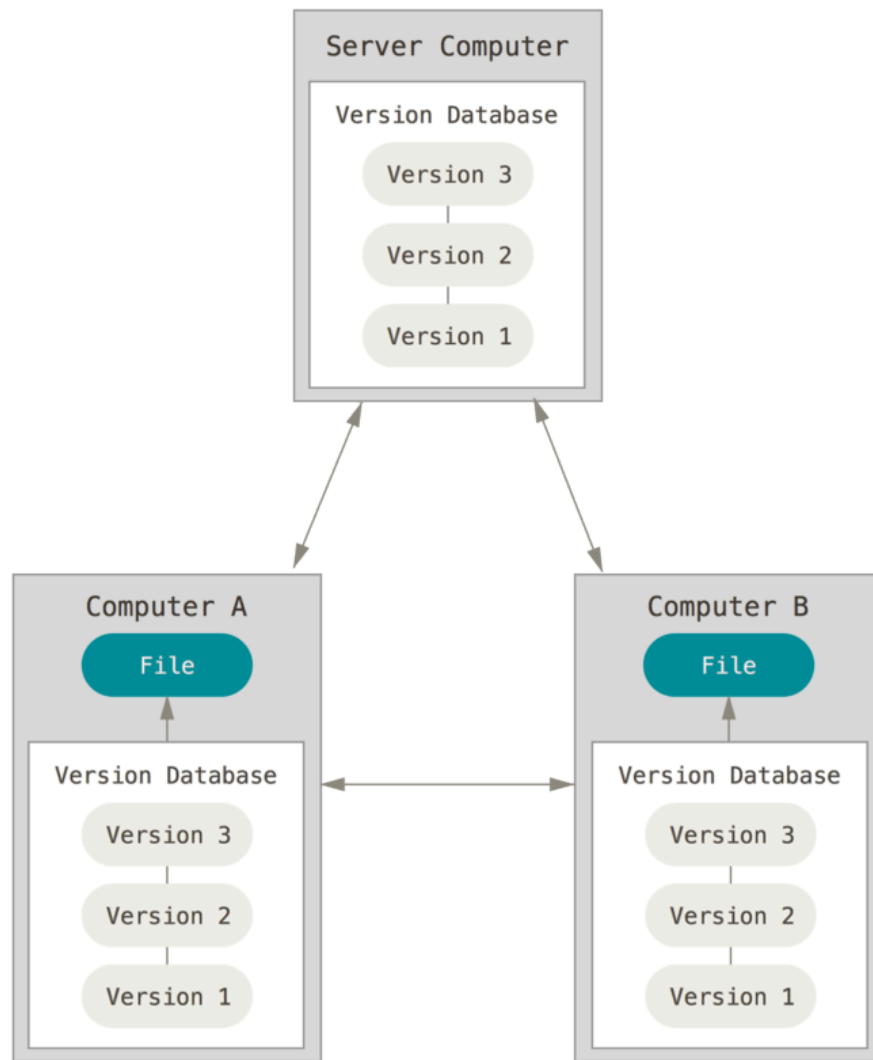
Centralized Version Control Systems

The next major issue that people encounter is that they need to collaborate with developers on other systems. To deal with this problem, Centralized Version Control Systems (CVCSs) were developed. These systems (such as CVS, Subversion, and Perforce) have a single server that contains all the versioned files, and a number of clients that check out files from that central place. For many years, this has been the standard for version control.



Distributed Version Control Systems

This is where Distributed Version Control Systems (DVCSs) step in. In a DVCS (such as Git, Mercurial, Bazaar or Darcs), clients don't just check out the latest snapshot of the files; rather, they fully mirror the repository, including its full history. Thus, if any server dies, and these systems were collaborating via that server, any of the client repositories can be copied back up to the server to restore it. Every clone is really a full backup of all the data.



CVCS vs DVCS

Setting Up Git on Local System

Step 1 : Check if git is already installed

```
git --version
```

Step 2 : If not installed download Git installer from <https://git-scm.com/>

Step 3 : Run installer and install git

Step 4 : Check if git is installed

```
git --version
```

Create GitHub Account

Step 1 : Go to <https://github.com/> and sign up for a new account

Step 2 : Login to GitHub

Step 3 : Create a new Repository (Private or Public)

Basic Git Commands

Step 1 : Create a new folder and open Git Bash/Cmd and go to the folder location

Step 2 : Run these commands for git configuration

```
git config --global user.email "sampleGitHub@email.com"
```

```
git config --global user.name "sampleGitHub_username"
```

Step 3 : Initialize git using this command

```
git init
```

Step 4 : Add some sample files in the folder by using this command

```
git touch <filename1.txt>
```

```
git touch <filename2.html>
```

```
git touch <filename3.py>
```

```
git touch <filename4.js>
```

Step 5 : Run these commands to check status, add files and commit your changes/updates

```
git status [ To check status ]
```

```
git add <filename> [ To add a particular file ]
```

```
git add . [ To add all the files ]
```

```
git commit -m "Commit Message" [ To commit all your changes ]
```

```
git remote add origin "github-url" [ To add remote repo link to local repo]
```

```
git push -u origin master [ To push your local changes to remote ]
```

Create a branch and add some files and make some changes and then add the branch to remote by :

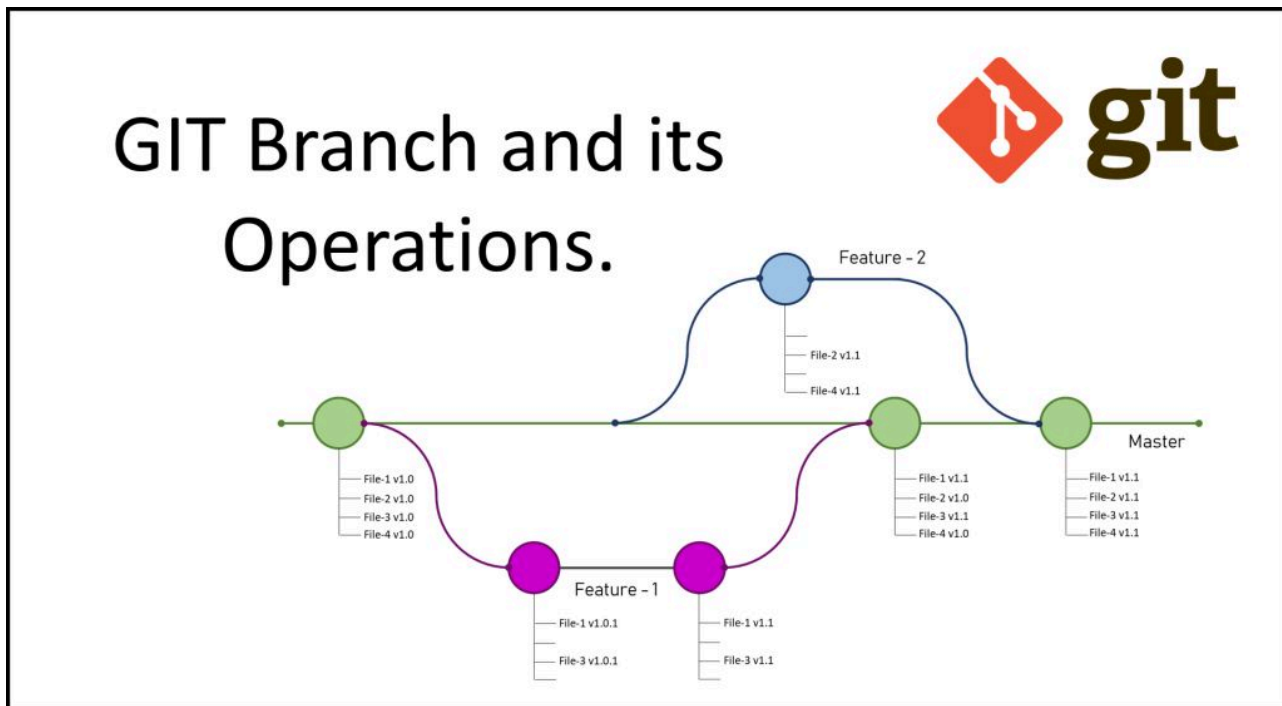
`git push -u origin "branch-name"`
`git clone "github url"`

[To push a particular branch to remote]
[To clone a GitHub Repo]

Git Branches

In this section you will be learning about :

- What are branches
 - How to create a branch in git repository
 - How to checkout to a particular branch in git repository
 - How to merge a particular branch to the master branch in git repository
 - How to delete a branch from local & remote repositories
- What are branches
 - Branches **allow you to develop features, fix bugs, or safely experiment with new ideas in a contained area of your repository**. You always create a branch from an existing branch. Typically, you might create a new branch from the default branch (**master**) of your repository.



Branching In Git

- How to create a branch in git repository
 - The **"git branch"** command can be used to create a new branch. When you want to start a new feature, you create a new branch off main using **"git branch new_branch"**.
- How to checkout to a particular branch in git repository
 - Once a branch is created you can then use **"git checkout new_branch"** to switch to that branch.
- How to merge a particular branch to the master branch in git repository
 - First we run **git checkout master** to change the active branch back to the master branch. Then we run the command **git merge new-branch** to merge the new feature into the master branch.

i **git merge** merges the specified branch into the currently active branch. So we need to be on the branch that we are merging into.

- **How to delete a branch from local & remote repositories**
 - To delete the local branch, just **run the git branch command again, this time with the -d (delete) flag, followed by the name of the branch you want to delete.**
 - You'll often need to delete a branch not only locally but also remotely. To do that, you use the following command: **git push <remote_name> --delete <branch_name>**. The branch still exists locally, if you haven't deleted it from your local repository.

Steps to follow while working with branches in Git

Step 1 : Create branch `git branch "branch_name"`

Step 2 : Checkout branch `git checkout "branch_name"`

Step 3 : Make some changes to your project

- **Add files, commit, push** `git push -u origin new_branch`
- **Check if the branch is visible in GitHub repository**

Step 4 : On local repo checkout to master branch `"git checkout master"`

Step 5 : Merge new branch in master branch `"git merge branch_name"`

Step 6 : Push all your changes `"git push -u origin master"`

Step 7 : Delete a particular branch

- `git branch -d "branch_name"` `// Will delete from local repository`
- `git push origin --delete "branch_name"` `// Will delete from remote repository`

Git Tags

A Tag in Git is a **reference that points to a specific point in Git history**. **Tagging** is generally used to capture a point in history that is used for **marking a version release (for example : v1.0)**. A tag is like a branch that doesn't change. Unlike branches, tags, after being created, have no further history of commits.

Step 1 : Checkout the branch where you want to create the tag

- `git checkout "branch_name"` `// Example : git checkout master`

Step 2 : Create a tag with some name

- `git tag "tag_name"` `// Example: git tag v1.0`
- `git tag -a v1.1 -m "tag for release ver 1.1"` `// Annotated Tag`

Step 3 : Display or Show tags

- `git tag` // Show all the tags
- `git show v1.0` // Show a particular tag
- `git tag -l "v1.*"` // Show all tags starting with v1

Step 3 : Push tags to remote repository

`git push origin v1.0`

- push all tags to remote : `git push --tags`

Step 4 : Delete tags (Only if required)

- To delete tags from local repository
 - `git tag -d v1.0`
 - `git tag --delete v1.0`
- To delete tags from remote repository
 - `git push origin -d v1.0`
 - `git push origin --delete v1.0`
 - `git push origin :v1.0`
- To delete multiple tags at one go
 - `git tag -d v1.0 v1.1` (local repository)
 - `git push origin -d v1.0 v1.1` (remote repository)

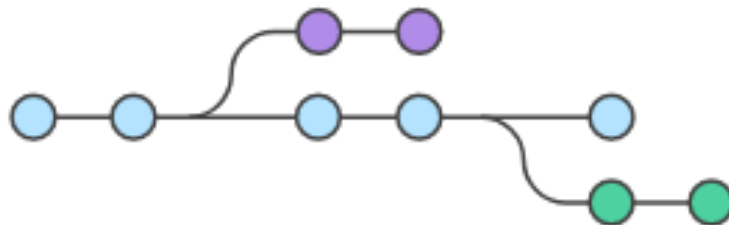
Checking out with tags

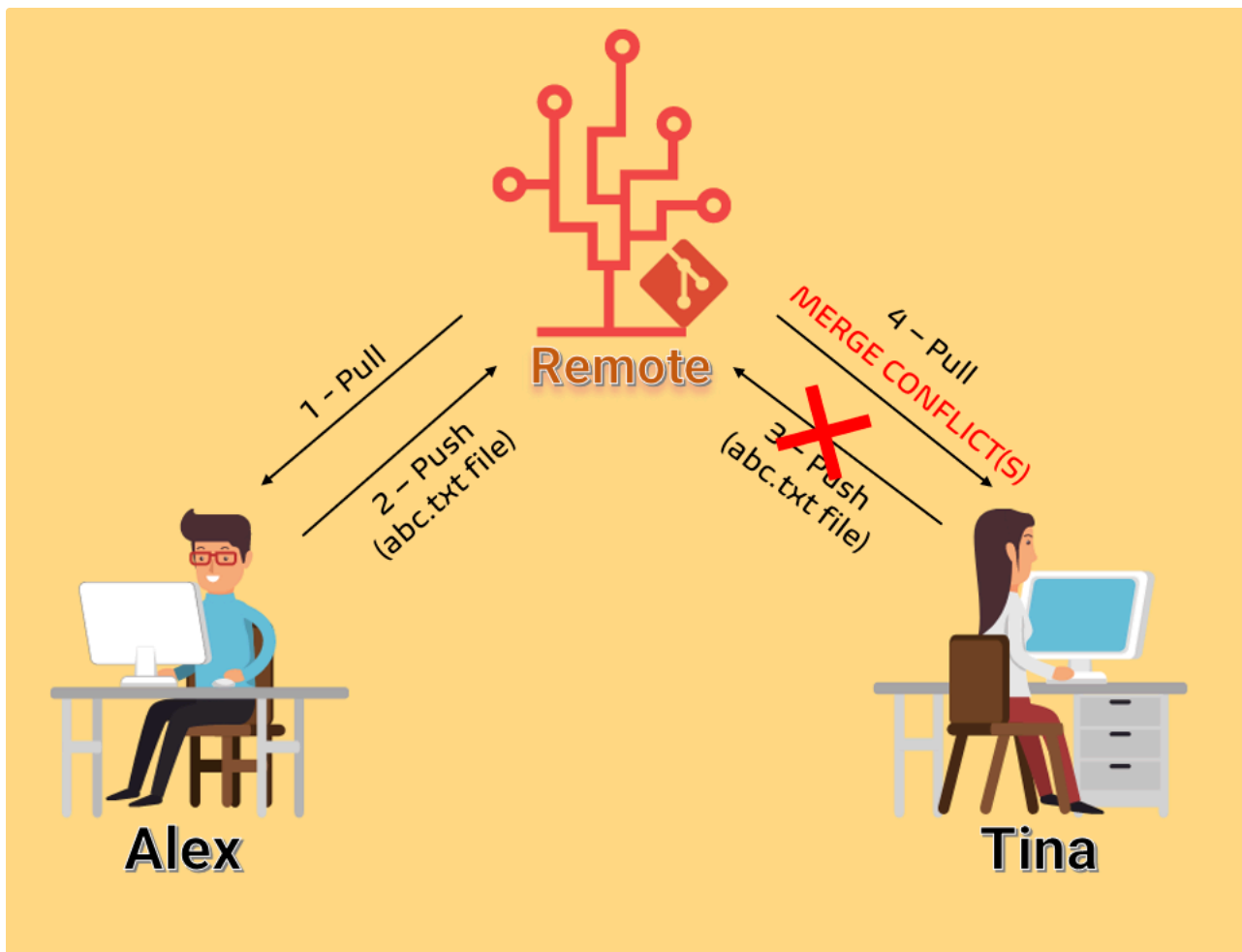
- We cannot checkout tags in Git
- We can create a branch from a tag and checkout the branch
 - `git checkout -b "branch_name" "tag_name"`
 - Example : `git checkout -b ReleaseVer1 v1.0`

Creating tags from past commits

- `git tag "tag_name" "reference of the past commit"`
- Example : `git tag v1.3 6gecf05`

Git Merge Conflicts





- Version control systems are all about managing contributions between multiple distributed authors (usually developers).
- Sometimes multiple developers may try to edit the same content.
- If Developer A tries to edit code that Developer B is editing a conflict may occur.
- To alleviate the occurrence of conflicts developers will work in separate **isolated branches**.
- The `git merge` command's primary responsibility is to combine separate branches and resolve any conflicting edits.

Understanding merge conflicts

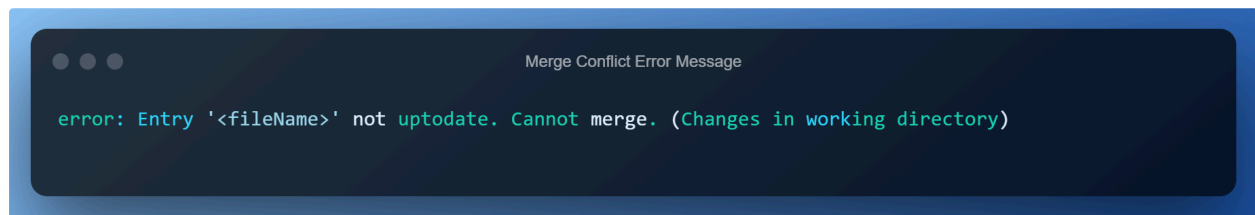
- Merging and conflicts are a common part of the Git experience.
- Conflicts in other version control tools like SVN can be costly and time-consuming.
- Git makes merging super easy.
- Most of the time, Git will figure out how to automatically integrate new changes.
- Conflicts generally arise when two people have changed the same lines in a file, or if one developer deleted a file while another developer was modifying it.
- In these cases, Git cannot automatically determine what is correct.
- Conflicts only affect the developer conducting the merge, the rest of the team is unaware of the conflict.
- Git will mark the file as being conflicted and halt the merging process.
- It is then the developers' responsibility to resolve the conflict.

Types of merge conflicts

A merge can enter a conflicted state at two separate points. When starting and during a merge process. The following is a discussion of how to address each of these conflict scenarios.

Git fails to start the merge

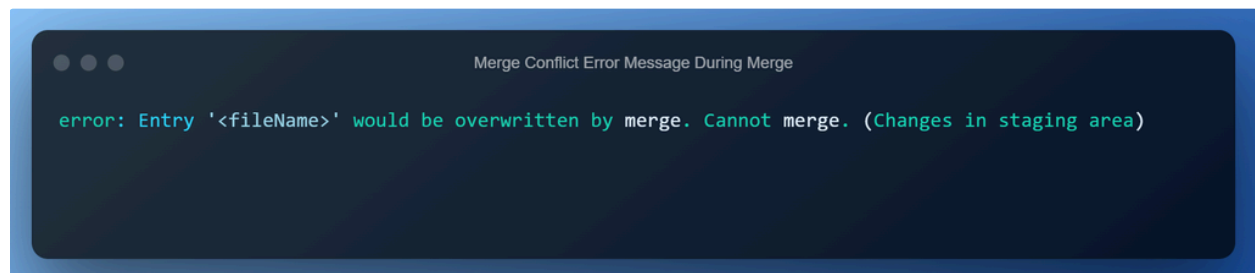
- A merge will fail to start when Git sees there are changes in either the working directory or staging area of the current project.
- Git fails to start the merge because these pending changes could be written over by the commits that are being merged in.
- When this happens, it is not because of conflicts with other developer's, but conflicts with pending local changes.
- The local state will need to be stabilized using `git stash`, `git checkout`, `git commit` or `git reset`. A merge failure on start will output the following error message:

A terminal window with a dark background and light blue title bar. The title bar has three dots on the left and the text "Merge Conflict Error Message" on the right. The terminal displays the following error message in green text: "error: Entry '<fileName>' not uptodate. Cannot merge. (Changes in working directory)".

```
error: Entry '<fileName>' not uptodate. Cannot merge. (Changes in working directory)
```

Git fails during the merge

- A failure DURING a merge indicates a conflict between the current local branch and the branch being merged.
- This indicates a conflict with another developers code.
- Git will do its best to merge the files but will leave things for you to resolve manually in the conflicted files.
- A mid-merge failure will output the following error message:

A terminal window with a dark background and light blue title bar. The title bar has three dots on the left and the text "Merge Conflict Error Message During Merge" on the right. The terminal displays the following error message in green text: "error: Entry '<fileName>' would be overwritten by merge. Cannot merge. (Changes in staging area)".

```
error: Entry '<fileName>' would be overwritten by merge. Cannot merge. (Changes in staging area)
```

How to identify merge conflicts

- As we have experienced from the proceeding example, Git will produce some descriptive output letting us know that a **CONFLICT** has occurred.
- We can gain further insight by running the `git status` command as shown below :

Identifying A Merge Conflict

```
$ git status
On branch main
You have unmerged paths.
(fix conflicts and run "git commit")
(use "git merge --abort" to abort the merge)

Unmerged paths:
(use "git add <file>..." to mark resolution)

both modified:   merge.txt
```

- The output from `git status` indicates that there are unmerged paths due to a conflict.
- The `merge.txt` file now appears in a modified state.
- Below is a sample merge conflict example :

Merge Conflict

```
$ cat merge.txt
<<<<<<< HEAD
this is some content to mess with
content to append
=====
totally different content to merge later
>>>>>>> new_branch_to_merge_later
```

How to Resolve Merge Conflicts in Git?

There are a few steps that could reduce the steps needed to resolve merge conflicts in Git.

1. The easiest way to resolve a conflicted file is to open it and make any necessary changes
2. After editing the file, we can use the git add a command to stage the new merged content
3. The final step is to create a new commit with the help of the git commit command
4. Git will create a new merge commit to finalize the merge

Let us now look into the Git commands that may play a significant role in resolving conflicts.

Git Commands to Resolve Conflicts

1. git log --merge

The git log --merge command helps to produce the list of commits that are causing the conflict

2. git diff

The git diff command helps to identify the differences between the states repositories or files

3. git checkout

The git checkout command is used to undo the changes made to the file, or for changing branches

4. git reset --mixed

The git reset --mixed command is used to undo changes to the working directory and staging area

5. git merge --abort

The git merge --abort command helps in exiting the merge process and returning back to the state before the merging began

6. git reset

The git reset command is used at the time of merge conflict to reset the conflicted files to their original state

 **Note :** We can also use the **Git Merge Tool** for resolving Merge Conflicts. Merge Conflicts can be resolved in different ways based on the user's convenience. But, prior to that, one should be familiar with the **git commands** required to resolve **Merge Conflicts**.

Experiment - 5

Introduction to Jenkins: What is Jenkins?, Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use

1. Introduction to Jenkins

Jenkins is an open-source automation server used for Continuous Integration and Continuous Deployment (CI/CD). It allows developers to automate repetitive tasks such as building, testing, and deploying software efficiently.

1.1 Features of Jenkins

- Open-source and widely used for CI/CD.
- Supports plugins for extensibility.
- Works with various build tools like Maven and Gradle.
- Can be deployed on local machines or cloud environments.
- Provides a web-based GUI for easy management.
- Allows job scheduling and pipeline execution.

1.2 Jenkins Architecture

- Master Node: Manages Jenkins configurations and distributes workloads to agents.
- Agent Node: Executes assigned jobs.
- Plugins: Extend Jenkins functionality.
- Pipelines: Define CI/CD workflows.
- Video Tutorial:

https://www.youtube.com/watch?time_continue=7&v=6YZvp2GwT0A&embeds_referri ng_euri=https%3A%2F%2Fdevopsbyanil.blogspot.com%2F&source_ve_path=MjM4NT E

2. Installing Jenkins on Local or Cloud Environment

2.1 Prerequisites

- Java (JDK 11 or newer)
- System requirements: Minimum 1GB RAM, 1GHz CPU
- Internet connection for downloading Jenkins and dependencies

2.2 Installing Jenkins on Local Machine (Windows/Linux/Mac)

Step 1: Install Java

1. Download and install Java JDK 11 or later.
2. Verify installation:
`java -version`

Step 2: Download and Install Jenkins

1. Download Jenkins from Jenkins official website.
2. Install Jenkins based on OS:
 - Windows: Run the .msi installer and follow the setup wizard.

Step 3: Start Jenkins

- Windows: Jenkins runs as a Windows service

Step 4: Access Jenkins Web Interface

1. Open a browser and go to:
`http://localhost:8080`
2. Retrieve the initial admin password:
`sudo cat /var/lib/jenkins/secrets/initialAdminPassword`
3. Paste the password in the browser prompt.

3. Configuring Jenkins for First Use

Step 1: Initial Setup

1. After logging in, Jenkins will prompt for plugin installation.
2. Select "Install suggested plugins".
3. Create an admin user with username, password, and email.

Step 2: Configure System Settings

1. Navigate to Manage Jenkins > Configure System.
2. Set up system-wide configurations such as Java path and build tool paths.

Step 3: Configure Nodes (Optional)

1. Go to Manage Jenkins > Manage Nodes and Clouds.
2. Add and configure additional agent nodes.

Step 4: Verify Installation

1. Create a sample job:
 - Go to New Item > Freestyle project.
 - Add a build step (e.g., Execute shell command echo "Hello Jenkins").
 - Click Save and Build Now.
2. Check the build logs for output.

Experiment-6

Practical Exercise: Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests.

Prerequisites

Before proceeding, ensure you have the following installed:

- **Java Development Kit (JDK) 8 or higher**
- **Jenkins** (Installed and configured on a local or cloud environment)
- **Maven** or **Gradle**
- **Git** (Installed and configured with a repository)

Step 1: Install Required Jenkins Plugins

To integrate Jenkins with Maven/Gradle and automate builds, install the following plugins:

1. Navigate to **Jenkins Dashboard > Manage Jenkins > Manage Plugins**.
2. Under the **Available Plugins** tab, search for and install:
 - **Pipeline Maven Plugin Database** (ID: pipeline-maven)
 - **Gradle Plugin** (ID: gradle)
 - **Git Plugin** (ID: git)
3. Restart Jenkins to apply the changes

Note: if the Plugins are not shown in Available Plugins and shown in installed plugin tab, then it indicates that plugins are already installed.

Step 2: Configure Maven/Gradle in Jenkins

For Maven:

1. Go to **Manage Jenkins > Global Tool Configuration**.
2. Scroll to the **Maven** section and click **Add Maven**.
3. Provide a **Name** (e.g., "Maven 3.8") and set the **Maven Home** directory path.
4. Click **Save**.

For Gradle:

1. Navigate to **Manage Jenkins > Global Tool Configuration**.
2. Scroll to the **Gradle** section and click **Add Gradle**.
3. Provide a **Name** (e.g., "Gradle 7.3") and set the **Gradle Home** path.
4. Click **Save**.

Step 3: Create a Jenkins Job

1. From the Jenkins dashboard, click **New Item**.
2. Enter a name for the project (e.g., "My-CI-Pipeline").
3. Select **Freestyle Project** or **Pipeline** (depending on your requirement).
4. Click **OK** to create the job.

Step 4: Configure Source Code Management (SCM) with Git

To pull the source code from a Git repository:

Example Project:

We will use a sample **Maven Java project** hosted on GitHub:

- Repository: <https://github.com/spring-projects/spring-petclinic.git>
1. In the job configuration page, navigate to **Source Code Management**.
 2. Select **Git**.
 3. Enter the repository URL:

`https://github.com/spring-projects/spring-petclinic.git`
 4. Under **Branch Specifier**, enter `*/main` (or `*/master` if applicable).
 5. If the repository is private, click **Add Credentials** and enter your GitHub username and access token.
 6. Click **Save**.

Step 5: Configure Build Steps

For Maven Projects:

1. Scroll to the **Build** section and click **Add Build Step > Invoke Top-Level Maven Targets**.
2. In the **Goals** field, enter:

`clean install`
3. Click **Save**.

For Gradle Projects:

1. Click **Add Build Step > Invoke Gradle Script**.
2. Select the **Gradle Version** configured earlier.
3. In the **Tasks** field, enter:

clean build

4. Click **Save**.

Step 6: Configure Post-Build Actions

1. Scroll to **Post-Build Actions**.
2. Click **Add Post-Build Action** > **Publish JUnit Test Result Report**.
3. In the **Test Report XMLs** field, enter:
4. target/surefire-reports/*.xml (for Maven)
build/test-results/test/*.xml (for Gradle)
5. Click **Save**.

Step 7: Trigger Automatic Builds

To trigger builds automatically upon code changes:

1. In the **Build Triggers** section, check **Poll SCM**.
2. Enter a schedule (e.g., H/5 * * * * to check for changes every 5 minutes).
3. Click **Save**.

Step 8: Run and Verify the Build

1. Navigate to the Jenkins dashboard and select the newly created job.
2. Click **Build Now**.
3. Monitor the build status in the **Build History** section.
4. Click on the latest build and navigate to **Console Output** to review logs.
5. Verify that the build completes successfully and test results are generated.

Experiment-7

Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook.

Objective:

To understand the basics of Ansible — Inventory, Playbooks, and Modules — and use Playbooks to automate server configurations. This includes setting up a Linux environment using Vagrant and VirtualBox, installed via Chocolatey.

What is Ansible?

Ansible is an open-source configuration management and automation tool. It helps system administrators and DevOps engineers to automate:

Software installation and configuration

Server provisioning

Application deployment

System updates and patching

Key Features of Ansible:

- Agentless: No agent is needed on the target machine. It uses SSH to communicate.
- Simple syntax: Uses YAML to define tasks in Playbooks.
- Idempotent: Running the same playbook multiple times won't cause unintended changes.
- Modular: Uses built-in modules like apt, yum, copy, service, etc.

Ansible Components:

Component	Description
Inventory	List of target machines to automate
Modules	Reusable scripts used to perform actions
Playbook	YAML file that contains tasks to be executed
Task	A single unit of work to be executed on a host

Part 1: Setting Up the Environment on Windows Using Chocolatey

- **Step 1: Install Chocolatey**

Open PowerShell as Administrator and run:

```
Set-ExecutionPolicy Bypass -Scope Process -Force; `
[System.Net.ServicePointManager]::SecurityProtocol =
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072; `
iex ((New-Object
System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))
```

Close and reopen PowerShell after installation.

- **Step 2: Install VirtualBox and Vagrant via Chocolatey**

```
choco install virtualbox -y
```

```
choco install vagrant -y
```

Part 2: Create and Configure a Linux VM using Vagrant

- **Step 3: Create a Project Directory**

```
mkdir ansible-vm
```

```
cd ansible-vm
```

- **Step 4: Initialize a Vagrant Project**

```
vagrant init ubuntu/bionic64
```

This creates a Vagrantfile.

- **Step 5: Edit the Vagrantfile**

Update the file to look like:

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"
  config.vm.network "forwarded_port", guest: 80, host: 8080
  config.vm.provider "virtualbox" do |vb|
    vb.memory = "2048"
    vb.cpus = 2
  end
end
```

- **Step 6: Start the Virtual Machine**

```
vagrant up
```

- **Step 7: SSH into the VM**

```
vagrant ssh
```

Part 3: Install Ansible inside the Linux VM

Once inside the VM:

```
sudo apt update
```

```
sudo apt install ansible -y
```

Verify:

```
ansible --version
```

Part 4: Write and Run Your First Ansible Playbook

- **Step 8: Create Inventory File**

Inside the VM, create inventory.ini:

```
[web]
```

```
127.0.0.1 ansible_connection=local
```

- **Step 9: Create a Playbook**

Create setup_webserver.yml:

```
---
```

```
- name: Setup Apache Web Server
  hosts: web
  become: yes
  tasks:
    - name: Install Apache
      apt:
        name: apache2
        state: present
        update_cache: yes
    - name: Enable and start Apache
      service:
        name: apache2
        state: started
        enabled: yes
    - name: Deploy custom HTML page
      copy:
        content: "<h1>Welcome to Ansible Web Server!</h1>"
        dest: /var/www/html/index.html
```

- **Step 10: Run the Playbook**

```
ansible-playbook -i inventory.ini setup_webserver.yml
```

Part 5: Verify the Web Server

✓ Check Web Server from inside VM:

`curl http://localhost`

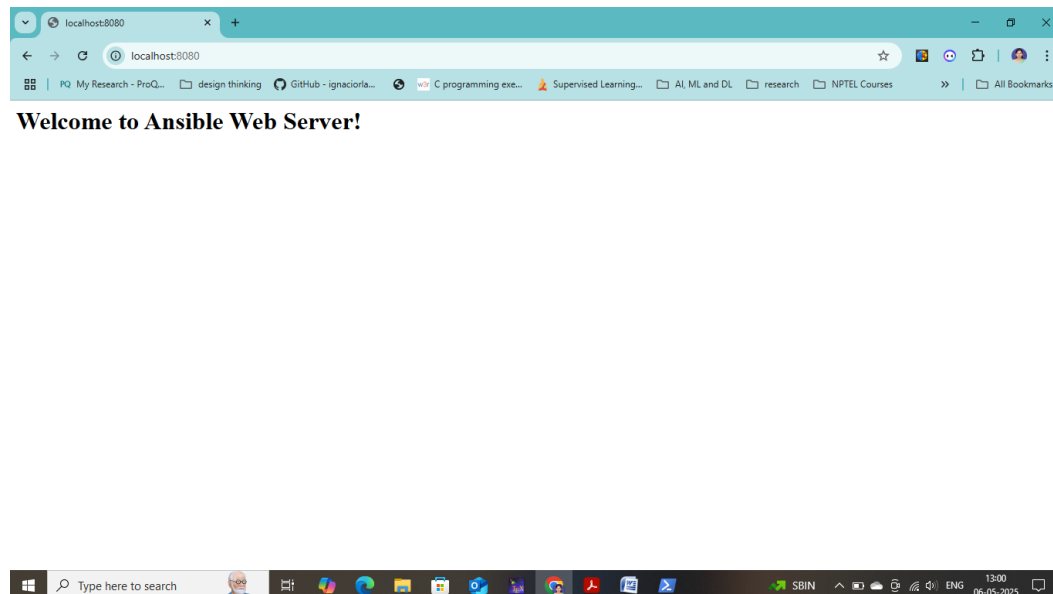
Expected output:

`<h1>Welcome to Ansible Web Server!</h1>`

```
vagrant@ubuntu-bionic:~$ ansible-playbook -i inventory.ini setup_webserver.yml
PLAY [Setup Apache Web Server] *****
TASK [Gathering Facts] *****
ok: [127.0.0.1]
TASK [Install Apache] *****
ok: [127.0.0.1]
TASK [Enable and start Apache] *****
ok: [127.0.0.1]
TASK [Deploy custom HTML page] *****
ok: [127.0.0.1]
PLAY RECAP *****
127.0.0.1 : ok=4 changed=0 unreachable=0 failed=0

vagrant@ubuntu-bionic:~$ curl http://localhost
<h1>Welcome to Ansible Web Server!</h1>vagrant@ubuntu-bionic:~$
```

Run the web server in browser using: <http://localhost:8080> and you can see the below output.



Conclusion:

In this experiment, you have:

Installed VirtualBox and Vagrant using Chocolatey

Created a Linux VM using Vagrant

Installed Ansible and learned its core components

Wrote and executed a basic Ansible Playbook

Deployed a simple web server automatically

Experiment-8

Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins.

Part 1: Install Jenkins and Maven on Ubuntu (VirtualBox)

Commands to get into ubuntu on VirtualBox

```
cd ansible-vm
```

```
vagrant up
```

```
vagrant ssh
```

Step 1: Update System

```
sudo apt update
```

```
sudo apt upgrade -y
```

Step 2: Install Java (required for Jenkins & Maven)

```
sudo apt install openjdk-17-jdk -y
```

```
java -version
```

Step 3: Install Maven

```
sudo apt install maven -y
```

```
mvn -version
```

Step 4: Install Jenkins

Add Jenkins Repository

```
wget -q -O - https://pkg.jenkins.io/debian/jenkins.io.key | sudo apt-key add -  
sudo sh -c 'echo deb https://pkg.jenkins.io/debian binary/ >  
/etc/apt/sources.list.d/jenkins.list'
```

Install Jenkins

```
sudo apt update
```

```
sudo apt install jenkins -y
```

Start and Enable Jenkins

```
sudo systemctl start jenkins
```



```
sudo systemctl enable jenkins
```

Access Jenkins

Open browser inside Ubuntu and go to:

```
http://localhost:8080
```

Unlock Jenkins

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Use this password to unlock Jenkins and install Suggested Plugins.

Part 2: Create a Simple "Hello World" Java Maven Project

Step 1: Create Directory and Project

```
mkdir hello-maven
```

```
cd hello-maven
```

```
mvn archetype:generate -DgroupId=com.example -DartifactId=hello-maven -  
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Step 2: Navigate to the Project

```
cd hello-maven
```

Step 3:

Edit the App.java

```
// File: src/main/java/com/example/App.java  
  
package com.example;  
public class App {  
    public static void main(String[] args) {  
        System.out.println("Hello, Jenkins CI!");  
    }  
}
```

Edit pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
        http://maven.apache.org/xsd/maven-4.0.0.xsd">  
    <modelVersion>4.0.0</modelVersion>  
    <groupId>com.example</groupId>  
    <artifactId>hello-maven</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
<properties>
  <maven.compiler.source>11</maven.compiler.source>
  <maven.compiler.target>11</maven.compiler.target>
</properties>
<dependencies>
  <!-- JUnit 4 for testing -->
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
<build>
  <plugins>
    <!-- Compiler plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.8.1</version>
      <configuration>
        <encoding>UTF-8</encoding>
        <source>11</source>
        <target>11</target>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

Edit AppTest.java

```
// File: src/test/java/com/example/AppTest.java
```

```
package com.example;
import org.junit.Test;
import static org.junit.Assert.*;
public class AppTest {
    @Test
    public void testApp() {
        assertTrue(true);
    }
}
```

Step 4: Build and Test

```
mvn clean install
```

Output will be in:

target/hello-maven-1.0-SNAPSHOT.jar

Part 3: Set Up Jenkins CI Pipeline for the Maven Project

Step 1: Open Jenkins → Create New Job

- Choose **Freestyle project**
- Name: hello-maven-ci

Step 2: Configure Project

Source Code Management: None (since local project)

Build:

- Add a build step: *Invoke top-level Maven targets*
- Goals: `clean install`
- POM: `path-to-your-project/pom.xml` (e.g., `workspace/hello-maven/hello-maven/pom.xml`)

Note: Keep All Jenkins Projects Inside `/var/lib/jenkins/workspace/`

Ex:

```
sudo cp -r /home/vidya/Desktop/hello-maven var/lib/jenkins/workspace/  
sudo chown -R jenkins:jenkins /var/lib/jenkins/workspace/hello-maven
```

Step 3: Save and Build

Click **Build Now** – Jenkins will compile and build the JAR locally.

Part 4: Deploy Artifact Using Ansible

Step 1: Install Ansible

```
sudo apt install ansible -y
```

Step 2: Prepare Ansible Playbook

Create a directory:

```
mkdir ansible-deploy
```

```
cd ansible-deploy
```

Create deploy.yml:

```
- name: Deploy Maven Artifact Locally
  hosts: localhost
  tasks:
    - name: Copy JAR to /opt/app
      copy:
        src: /home/user/hello-maven/target/hello-maven-1.0-SNAPSHOT.jar
        dest: /opt/app/hello-maven.jar
```

Note: Change the src path accordings

Step 3: Run the Playbook

```
sudo mkdir -p /opt/app
```

```
ansible-playbook deploy.yml
```

Experiment-9

Introduction to Azure DevOps: Overview of Azure DevOps Services, Setting Up an Azure DevOps Account and Project

Introduction to Azure DevOps

What is Azure DevOps?

Azure DevOps is a cloud-based service from Microsoft that provides developer services to support teams in planning work, collaborating on code development, and building & deploying applications.

Azure DevOps Services – Overview

Azure DevOps offers the following main services:

1. Azure Repos

- Provides **Git repositories** or **Team Foundation Version Control (TFVC)**.
- Used for **source code version control**.
- Supports **pull requests, branching, and code reviews**.

2. Azure Pipelines

- Used for **Continuous Integration (CI)** and **Continuous Delivery (CD)**.
- Supports building, testing, and deploying code automatically to any platform.

3. Azure Boards

- Agile planning tools: **Kanban boards, backlogs, and dashboards**.
- Helps manage **tasks, bugs, and user stories**.

4. Azure Test Plans

- Provides **manual and exploratory testing tools**.
- Helps ensure software quality and coverage.

5. Azure Artifacts

- Manages **package dependencies** (e.g., NuGet, npm, Maven).
 - Allows teams to **create, host, and share packages**.
-

Setting Up an Azure DevOps Account and Project

Step 1: Create a Microsoft Account (if not already available)

- Visit: <https://signup.live.com>
- Use this account to sign in to Azure DevOps.

Step 2: Sign in to Azure DevOps

- Go to: <https://dev.azure.com>
- Use your Microsoft credentials to log in.

Step 3: Create a New Organization

- An **organization** in Azure DevOps groups related projects.
- Click on "**New Organization**", provide a name, select a region.

Step 4: Create a Project

- Inside your organization, click "**New Project**".
- Fill in:
 - **Project name**
 - **Visibility**: Public or Private
 - Choose **Version Control**: Git or TFVC
 - Select **Work Item Process**: Agile, Scrum, CMMI, Basic

Step 5: Explore Services

- After project creation, you can start using:
 - **Repos** for code
 - **Boards** for task tracking
 - **Pipelines** for CI/CD
 - **Artifacts** for packages
 - **Test Plans** for testing

Experiment-10

Creating Build Pipelines: Building a Maven/Gradle Project with Azure Pipelines, Integrating Code Repositories (e.g., GitHub, Azure Repos), Running Unit Tests and Generating Reports

Objective:

To configure a self-hosted agent on a local machine, build a Maven or Gradle project using Azure DevOps Pipelines, run unit tests, and generate reports — all without incurring any Azure charges.

Theory Overview

What is a Self-Hosted Agent?

A **Self-hosted agent** is a personal or institutional computer (e.g., your own laptop or lab PC) that runs Azure Pipelines jobs instead of using Microsoft-hosted (cloud) agents.

- **No billing or time limits**
- **Full control** over environment
- Great for **offline or classroom scenarios**

Steps to Build a Maven/Gradle Project with a Self-Hosted Agent

Step 1: Prepare Your System (Self-Hosted Agent)

You'll need:

- A PC with Java, Maven or Gradle installed
- Git installed
- Internet access to download Azure agent

Step 2: Create an Azure DevOps Project (One-Time Setup by Instructor)

1. Go to <https://dev.azure.com>
2. Sign in and create a **new organization** (e.g., college-devops-org)
3. Create a **project** (e.g., BuildPipelineLab)

Step 3: Register Your Self-Hosted Agent

1. In Azure DevOps portal:

- Go to **Project Settings > Agent Pools**
 - Click **Default** pool
 - Select **New Agent** > Choose OS (Windows/Linux/macOS)
2. Follow the steps shown to:
- Download agent package
 - Extract it to a folder, e.g., C:\azagent
 - Run these commands:

```
cd C:\azagent
config.cmd
```

3. Provide:
- DevOps URL
 - Personal Access Token (PAT)
 - Agent name (e.g., labagent1)
 - Choose "**run as a service**" if prompted
4. Start the agent:

```
run.cmd
```

Step 4: Push Code to Repository

1. Create a repository in Azure Repos or GitHub
2. Push your Maven/Gradle project there

Step 5: Create `azure-pipelines.yml` File in Your Repo

✔ For Maven:

```
trigger:
- main

pool:
  name: Default # Refers to your self-hosted agent pool

steps:
- task: Maven@3
  inputs:
    mavenPomFile: 'pom.xml'
    goals: 'clean install'
    publishJUnitResults: true
```

✔ For Gradle:

```
trigger:
- main
```



```
pool:
  name: Default

steps:
- script: ./gradlew clean build
  displayName: 'Build with Gradle'
```

Step 6: Run the Pipeline

1. Commit the YAML file to your repository.
2. Azure Pipelines will trigger automatically when changes are pushed.
3. The job will run on your own PC or lab machine (self-hosted agent).

Step 7: View Unit Test Reports

- Go to **Pipelines > Runs**
- Click the latest run
- Go to **Tests** tab to view results

If you don't see test results:

- Make sure you're using a testing framework like JUnit/TestNG.
- Ensure reports are being generated in standard format (e.g., `target/surefire-reports` for Maven).

Advantages of Using Self-Hosted Agent

- **Zero cost** – no billing risk
- Use lab computers or personal laptops
- More control over software environment
- Ideal for educational institutions

Exercise-11

Creating Release Pipelines: Deploying Applications to Azure App Services, Managing Secrets and Configuration with Azure Key Vault, Hands-On: Continuous Deployment with Azure Pipelines

Objective

To understand how to deploy applications to Azure App Services using Azure Pipelines, manage secrets using Azure Key Vault, and implement continuous deployment.

1. Deploying Applications to Azure App Services

Azure App Services Overview

Azure App Services is a fully managed platform for building, deploying, and scaling web apps.

Steps to Deploy using Azure Pipelines

1. Create an App Service in Azure:

- Go to Azure Portal → Create a Resource → Web App.
- Provide details like App name, Runtime stack, Region, and Resource group.

2. Create a Release Pipeline in Azure DevOps:

- Go to Azure DevOps → Pipelines → Releases.
- Click on New Pipeline → Select Azure App Service Deployment template.

3. Configure the Artifact:

- Link the build artifact from your build pipeline (Maven/Gradle).
- Choose the appropriate version and source.

4. Add a Stage and Deploy:

- Select the App Service target.
- Configure App Service connection and app name.

5. Trigger Deployment:

- Enable continuous deployment trigger.
 - Save and create a release.
-

2. Managing Secrets and Configuration with Azure Key Vault

What is Azure Key Vault?

Azure Key Vault is a cloud service to securely store and access secrets, keys, and certificates.

Using Key Vault in Azure Pipelines

1. Create a Key Vault:

- Go to Azure Portal → Create Key Vault.
- Store secrets like DB passwords, API keys.

2. Configure Access:

- Assign appropriate access policies (for Azure Pipelines or App Service).

3. Link Key Vault in Azure Pipeline:

- In the pipeline YAML or classic editor:
variables:
 - group: MyKeyVaultSecrets
- Use secrets like \$(MySecret) in scripts or tasks.

4. Integrate with App Service Configuration:

- App Services can directly pull secrets using Managed Identity.

3. Hands-On: Continuous Deployment with Azure Pipelines

What is Continuous Deployment (CD)?

Continuous Deployment is the practice of automatically deploying every code change that passes the CI pipeline.

Setup CD with Azure Pipelines:

1. Ensure CI Pipeline is Configured:

- Generates build artifact after every commit.

2. Create a Release Pipeline:

- Link the artifact from CI.
- Add deployment stages (Dev, QA, Prod).

3. Enable Continuous Deployment Trigger:

- In Artifact section → enable CD trigger.

4. Deploy to Azure App Service:

- Use the Azure App Service Deploy task.
- Set environment-specific settings.

5. Monitor and Validate:

- Check deployment status.
- Validate on the App Service URL.

Expected Outcome

- You will be able to:
 - Automatically deploy your Java web app to Azure App Services.
 - Use Azure Key Vault to manage sensitive data securely.
 - Configure full CI/CD with Azure Pipelines.

Exercise-12

Practical Exercise and Wrap-Up: Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A.

Objective:

To integrate the knowledge gained from previous experiments by designing and executing a complete DevOps pipeline for a Java-based application. This includes stages such as version control, build automation, testing, packaging, deployment, and optional containerization and monitoring.

Part A: Practical Hands-On Exercise

1. Project Setup

- **Choose a Java Application:** You can use a sample application (e.g., a Spring Boot app or a basic Java servlet).
 - **Version Control:**
 - Create a new GitHub repository.
 - Push the project code to the repository.
 - Ensure `.gitignore` excludes `target/`, `.idea/`, and other unnecessary files.
-

2. Build Automation with Maven or Gradle

- Create or validate your `pom.xml` (Maven) or `build.gradle` (Gradle) file.
- Include:
 - Project dependencies
 - Build plugins
 - Test configuration
- **Example (for Maven):**

```
<build>
  <plugins>
```

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.1</version>
  <configuration>
    <source>11</source>
    <target>11</target>
  </configuration>
</plugin>
</plugins>
</build>
```

3. Configure Jenkins

- **Install Jenkins (local/cloud – AWS EC2 preferred)**
 - **Install Plugins:**
 - Git Plugin
 - Maven Integration Plugin
 - Docker (if using containerization)
 - **Create a Jenkins Job (Freestyle or Pipeline):**
 - Source Code Management: GitHub Repo
 - Build Trigger: "Poll SCM" or Webhook from GitHub
 - Build Steps:
 - `mvn clean install`
 - Optional: Run test cases, create artifacts
-

4. Unit Testing

- Write unit test cases using JUnit or TestNG.
- Ensure Jenkins is configured to fail builds on test failures.
- **Sample Test (JUnit):**

```
@Test
public void testAdd() {
    assertEquals(5, Calculator.add(2, 3));
}
```

5. Artifact Management

- **Local Option:** Save the .jar or .war file in Jenkins workspace or shared location.
 - **Advanced Option:** Use Nexus/Artifactory to store the generated build artifacts.
-

6. Containerization (Optional but Encouraged)

- Create a Dockerfile:
 - FROM openjdk:11
 - COPY target/myapp.jar myapp.jar
 - ENTRYPOINT ["java", "-jar", "myapp.jar"]
 - Build Docker Image:
 - docker build -t yourusername/myapp:v1 .
 - Push to Docker Hub:
 - docker push yourusername/myapp:v1
-

7. Deployment

- **Local Deployment:**
 - Run the .jar or use Docker: `docker run -p 8080:8080 yourusername/myapp:v1`
- **Cloud Deployment:**
 - SCP the artifact to AWS EC2 or use Jenkins to automate deployment.
 - Start the application on the cloud server.